CLARA

Complaint-Led Aggregation for Road Anomalies

Federated Learning con aggregazione pesata da segnale NLP
per il monitoraggio distribuito del degrado stradale urbano

Nicola Frugieri
Deep Learning & Natural Language Processing — Progetto combinato
Università degli Studi di Salerno
github.com/nf0101/CLARA

10 novembre 2026

Sommario

Il presente lavoro propone e valuta **CLARA** (*Complaint-Led Aggregation for Road Anomalies*), un sistema di *Federated Learning* in cui i pesi con cui il server combina gli aggiornamenti dei client sono modulati da un segnale di urgenza estratto, tramite una pipeline NLP, da lamentele testuali dei cittadini. La strategia di aggregazione che realizza questo meccanismo è denominata NLP-Weighted, ed è con questo nome che compare nel confronto sperimentale con FedAvg e con la variante a oracolo. Il caso d'uso è il monitoraggio distribuito del degrado stradale urbano, con particolare attenzione alla classe critica D40 (*pothole*), definita in modo esogeno sulla base della classificazione dei danni stradali consolidata nella letteratura di ingegneria delle infrastrutture. La pipeline NLP combina un modello NER pre-addestrato (spaCy) con una tabella di mapping località→client, un modulo sentiment ibrido basato su un Transformer Twitter-RoBERTa e su euristiche lessicali di dominio, e un aggregatore che produce il vettore di urgenza per client normalizzato.
L'esperimento è strutturato secondo un protocollo di validazione a tre componenti (criticità esogena, upper bound informativo Oracle, sweep controllato sul rumore NLP) progettato per evitare la circolarità inerente al confronto di un metodo NLP-informato con una metrica NLP-derivata. Su un dataset *Non-IID* severo (RDD2022, partizionato country-wise sui 6 paesi del challenge CRDDC), NLP-Weighted supera FedAvg del **15%** su critical_ap50 pur mantenendo performance comparabili sulle metriche generali. La curva di robustezza al rumore NLP mostra un plateau fino ad accuratezza NLP simulata del 70%, con degradazione solo al 50%. Un'analisi di sensitivity sugli iperparametri di aggregazione conferma la stabilità del metodo.

Parole chiave: Federated Learning, Object Detection, YOLO, NLP, Named Entity Recognition, Sentiment Analysis, Non-IID.

Indice

1 Introduzione **4**
- 1.1 Contesto e motivazione . . . 4
- 1.2 Il problema metodologico . . . 5
- 1.3 Contributo del lavoro . . . 5
- 1.4 Struttura del documento . . . 6

2 Lavori correlati **6**
- 2.1 Federated Learning e Non-IID . . . 7
- 2.2 Road Damage Detection . . . 7
- 2.3 NLP e civic reporting per smart city . . . 8
- 2.4 Posizionamento del contributo . . . 10

3 Metodo **11**
- 3.1 Architettura ad alto livello . . . 11
- 3.2 Modulo Federated Learning . . . 12
  - 3.2.1 Rilevatore di base . . . 12
  - 3.2.2 Protocollo federato . . . 12
  - 3.2.3 FedAvg (*baseline context-blind*) . . . 13
  - 3.2.4 Oracle-Weighted . . . 13
  - 3.2.5 NLP-Weighted . . . 13
  - 3.2.6 Provider disponibili . . . 15
- 3.3 Modulo Natural Language Processing . . . 15
  - 3.3.1 Stadio 1 — NER geografico . . . 16
  - 3.3.2 Stadio 2 — Mapping località→client . . . 16
  - 3.3.3 Stadio 3 — Severity classification ibrida . . . 17
  - 3.3.4 Stadio 4 — Aggregazione per client . . . 18
- 3.4 Integrazione dei due moduli . . . 19

4 Setup sperimentale **19**
- 4.1 Dataset: RDD2022 . . . 19
- 4.2 Partizionamento Non-IID: client come paesi . . . 20
- 4.3 Framework tecnologico . . . 21
- 4.4 Hardware ed esecuzione . . . 21
- 4.5 Metriche di valutazione . . . 22
- 4.6 Dataset testuale di lamentele sintetiche . . . 22
- 4.7 Configurazione dello sweep . . . 23

5 Protocollo di validazione **24**
- 5.1 Il problema della circolarità . . . 24
- 5.2 Componente A: definizione esogena della classe critica . . . 25
- 5.3 Componente B: upper bound informativo con Oracle . . . 25
- 5.4 Componente C: sweep controllato sul rumore NLP . . . 26

5.5 Il ruolo della pipeline NLP reale . . . 26
5.6 Sintesi: le tre *research question* . . . 27

6 Risultati **27**
6.1 RQ1: confronto delle strategie . . . 27
6.2 RQ2: robustezza al rumore NLP . . . 29
6.3 Ablation: sensitivity agli iperparametri di aggregazione . . . 31
6.4 Prova del concetto: pipeline NLP reale end-to-end . . . 33

7 Discussione **34**
7.1 Cosa dimostrano i risultati . . . 34
7.2 *Effect size* e prospettiva . . . 35
7.3 Limiti riconosciuti . . . 35
7.4 Lezioni apprese . . . 37

8 Lavori futuri **38**
8.1 Integrazione con baseline Non-IID-aware . . . 38
8.2 Sostituzione del dataset testuale con lamentele reali . . . 39
8.3 Deployment a scala intra-città . . . 40
8.4 Ampliamento della statistica sperimentale . . . 40
8.5 Scala full di training . . . 41
8.6 Riepilogo delle direzioni . . . 41

9 Conclusioni **42**

A Iperparametri completi degli esperimenti **45**
A.1 Iperparametri comuni a tutte le *run* . . . 45
A.2 Iperparametri specifici per strategia . . . 45
A.2.1 FedAvg . . . 45
A.2.2 Oracle-Weighted . . . 46
A.2.3 NLP-Weighted . . . 46
A.2.4 `NoisyOracleUrgencyProvider` (usato in RQ2) . . . 46
A.3 Iperparametri della pipeline NLP . . . 46

B Statistiche complete del dataset **47**
B.1 RDD2022: distribuzione per client . . . 47
B.2 Dataset di lamentele: distribuzione . . . 48

C Tabelle complete dello sweep sperimentale **49**
C.1 RQ1: valori individuali per seed . . . 50
C.2 RQ2: valori individuali per seed e livello di rumore . . . 50
C.3 Ablation: valori per configurazione . . . 51
C.4 Prova del concetto: dettaglio della *run* . . . 51
C.5 Valutazione del modulo Sentiment/Severity . . . 51
C.6 Note sulla varianza stocastica . . . 52

1 Introduzione

Il monitoraggio del degrado delle infrastrutture stradali è un problema di rilevanza crescente per le città moderne. Buche, fessure e cedimenti del manto stradale rappresentano un rischio per la sicurezza dei veicoli, in particolare per motociclette e biciclette, e generano costi diretti (danni ai veicoli, incidenti) e indiretti (riduzione della qualità della mobilità urbana). La rilevazione automatica di questi difetti a partire da immagini acquisite da veicoli in movimento è un problema di *object detection* attivo in letteratura, e ha motivato la costruzione di dataset dedicati come RDD2022 [1], distribuito nell'ambito del *Crowdsensing-based Road Damage Detection Challenge* (CRDDC) [2].

Una peculiarità del problema, che questo lavoro sfrutta esplicitamente, è che i dati per la sua soluzione sono naturalmente *distribuiti*. Le immagini di potenziale interesse arrivano da flotte di veicoli sparse per il territorio, ciascuna con caratteristiche statistiche differenti (clima, tipologia di manto stradale, densità di traffico, tipologia di veicoli utilizzati per l'acquisizione). Il *Federated Learning* (FL) [3] è la famiglia di tecniche progettata proprio per addestrare modelli condivisi su dati che restano distribuiti sui rispettivi client, evitando sia il trasferimento centralizzato dei dati (con i relativi vincoli di banda e privacy) sia la duplicazione degli sforzi di addestramento locale.

1.1 Contesto e motivazione

FL nella sua formulazione standard (FedAvg, [3]) aggrega gli aggiornamenti dei client proporzionalmente al volume dei loro dati locali. È la scelta più naturale e la baseline standard di ogni sistema FL, ma ha un limite noto in scenari *Non-IID*, in cui i dati locali dei client differiscono statisticamente tra loro: metodi come FedProx [4] e SCAFFOLD [5] sono stati proposti per gestire in modo più accurato il *client drift* che ne risulta.

Un aspetto ortogonale, meno esplorato nella letteratura FL, riguarda la possibilità di usare informazione *esterna* al training locale per modulare l'aggregazione. Nel caso d'uso del monitoraggio urbano, un canale di informazione naturalmente disponibile e complementare ai dati visuali è costituito dalle *segnalazioni testuali dei cittadini*. Le piattaforme di *civic reporting* (portali comunali, applicazioni mobili, servizi di *crowdsourcing*) raccolgono quotidianamente descrizioni in linguaggio naturale di problemi infrastrutturali, con informazioni sulla loro localizzazione geografica e con un tono che riflette la percezione della gravità. Questa informazione non può essere estratta dai soli dati visuali: nessun modello di *object detection*, per quanto accurato, può distinguere una buca "fastidiosa ma tollerabile" da una "pericolosa per la sicurezza" basandosi solo sull'immagine.

Da qui l'intuizione del presente lavoro: usare l'output di una pipeline NLP che processa le lamentele cittadine come segnale contestuale per modulare l'aggregazione FL. Il canale testuale fornisce al server FL un segnale di *urgenza* per ciascun client, che indica quanto le aree di competenza di quel client siano percepite come critiche. Il server darà più peso agli aggiornamenti dei client con urgenza elevata, concentrando la capacità del modello sulle anomalie a maggior rischio.

Task NLP e dominio di applicazione. Rispetto alla classificazione dei task di *Natural Language Processing*, il presente lavoro si colloca nell'ambito della **sentiment analysis**, declinata come *severity classification* di dominio: il modulo non stima una polarità generica, ma un grado di urgenza infrastrutturale su scala continua. Il *Named Entity Recognition* geografico interviene come stadio di supporto, necessario ad associare ciascuna segnalazione al client federato competente. Il dominio di applicazione è il *civic reporting* per il monitoraggio urbano, e la problematica specifica che il lavoro caratterizza è il divario fra *sentiment* generico e *severity* di dominio (sezione 3.3.3), acuito dal fatto che le segnalazioni cittadine costituiscono un *proxy* distorto della distribuzione oggettiva delle anomalie (sezione 2.3).

1.2 Il problema metodologico

Formulata così, l'idea presenta un problema metodologico non banale. Valutare l'efficacia di un meccanismo di aggregazione NLP-informato richiede di definire una nozione di "classe critica" su cui misurare il miglioramento. Se questa nozione è derivata dall'output del segnale NLP stesso, il ragionamento diventa circolare: il metodo appare superiore perché la metrica di riferimento coincide, per costruzione, con ciò che il metodo ottimizza. Se invece è derivata dai dati del sistema FL, si rischia di misurare l'allineamento tra i pesi di aggregazione e la distribuzione della classe critica anziché l'effetto del meccanismo.

Questo problema è affrontato nel presente lavoro tramite un protocollo di validazione a tre componenti (sezione 5). Innanzitutto, la classe critica è definita in modo *esogeno* tramite la letteratura di ingegneria delle infrastrutture stradali, indipendentemente sia dall'output NLP sia dai dati FL. In secondo luogo, viene introdotto un *upper bound informativo* costituito da un provider Oracle che usa la distribuzione della classe critica direttamente dal dataset, simulando un modulo NLP idealmente perfetto. Infine, un provider sintetico rumoroso consente di variare in modo controllato l'accuratezza del segnale NLP e di caratterizzare empiricamente la robustezza del sistema al rumore.

1.3 Contributo del lavoro

Il presente lavoro apporta quattro contributi principali. Nel seguito **CLARA** indica il sistema nel suo insieme — pipeline NLP, modulo federato e protocollo di validazione — mentre NLP-Weighted indica la specifica strategia di aggregazione che ne realizza il meccanismo, ed è il nome con cui compare nelle tabelle sperimentali.

1. NLP-Weighted: un meccanismo di aggregazione NLP-informato. Estendiamo FedAvg con un meccanismo di pesatura contestuale che combina la componente volume-based tradizionale con un segnale esterno di urgenza per client. Il meccanismo è formalizzato in sezione 3.2.5, con due salvaguardie (*cap* e *time_decay*) contro l'*oblio catastrofico*.

2. Protocollo di validazione a tre componenti. Definiamo e giustifichiamo un protocollo per valutare metodi FL guidati da segnali di dominio esterni, evitando la circolarità tipica di questo tipo di valutazioni. Il protocollo è presentato come contributo indipendente e trasferibile in sezione 5.

3. Pipeline NLP end-to-end per lamentele cittadine. Implementiamo una pipeline modulare a quattro stadi (NER geografico, mapping località→client, sentiment/severity ibrida, aggregazione per client) che processa lamentele testuali e produce un vettore di urgenza compatibile con il modulo FL tramite un'interfaccia astratta comune. La pipeline è descritta in dettaglio in sezione 3.3 ed è disponibile come codice aperto.[1]

4. Caratterizzazione empirica su RDD2022. Riportiamo uno sweep sperimentale sistematico di 51 *run* federate indipendenti sul dataset RDD2022 partizionato *country-wise*, strutturato in tre fasi (RQ1: confronto strategie; RQ2: robustezza al rumore; RQ3: sensitivity iperparametri) e completato da una dimostrazione *end-to-end* con la pipeline NLP reale. I risultati principali sono un miglioramento del +15% su critical_ap50 rispetto a FedAvg, un plateau di robustezza fino al 70% di accuratezza NLP simulata, e la stabilità agli iperparametri di salvaguardia.

Collocazione nelle linee guida del corso. Nella classificazione proposta dal corso di Natural Language Processing, il progetto si colloca nell'ambito dell'**Obiettivo Ricerca** con approccio **tradizionale**: l'avanzamento proposto non riguarda una singola fase della pipeline, ma tre. Sulla *caratterizzazione del problema*, formula la nozione di canale esogeno al training federato come dimensione sotto-esplorata nella letteratura FL. Sulla *metodologia*, la concretizza in una pipeline NLP disaccoppiata dal rilevatore tramite un'interfaccia astratta comune. Sulla *valutazione sperimentale*, introduce un protocollo a tre componenti progettato per rompere la circolarità intrinseca a questo tipo di valutazioni. L'avanzamento rivendicato è di natura **metodologica**: i singoli ingredienti — pesatura dell'aggregazione, *sentiment* ibrido, NER per la localizzazione geografica — esistono già in letteratura, e il guadagno empirico riportato in sezione 6 è modesto in valore assoluto (sezione 7.2). Il contributo sta nella loro composizione coerente attorno a un problema metodologico definito, non nell'entità del miglioramento misurato.

1.4 Struttura del documento

Il resto del documento è organizzato come segue. Sezione 2 inquadra il lavoro rispetto alla letteratura su FL, *road damage detection* e NLP per applicazioni di *smart city*. Sezione 3 descrive il metodo, con particolare attenzione al modulo FL (sezione 3.2), al modulo NLP (sezione 3.3) e all'integrazione tra i due (sezione 3.4). Sezione 4 presenta il dataset, il framework e la configurazione dello sweep sperimentale. Sezione 5 formalizza il protocollo di validazione a tre componenti. Sezione 6 riporta i risultati sperimentali; sezione 7 li interpreta e ne discute i limiti; sezione 8 traccia le direzioni di estensione. Le conclusioni sono in sezione 9. Il documento include tre appendici con dettagli sugli iperparametri (sezione A), le statistiche complete del dataset (sezione B) e le tabelle numeriche integrali dello sweep (sezione C).

2 Lavori correlati

Il presente lavoro si colloca all'intersezione di tre filoni di ricerca distinti: il *Federated Learning* per applicazioni di *computer vision*, la rilevazione automatica del degrado stradale, e l'analisi

[1] Repository: https://github.com/nf0101/DLNLP.

NLP di segnalazioni testuali dei cittadini nelle piattaforme di *civic reporting*. Discutiamo di seguito i lavori principali dei tre filoni e chiudiamo con il posizionamento del contributo di questo lavoro rispetto ad essi.

2.1 Federated Learning e Non-IID

Il paradigma del *Federated Learning* è stato introdotto da McMahan et al. [3] con l'algoritmo FedAvg, che combina la discesa del gradiente locale in parallelo su più client con un'aggregazione centralizzata dei pesi proporzionale al volume di dati locali. FedAvg funziona bene quando i dati locali sono approssimativamente IID, ma degrada sensibilmente in scenari *Non-IID* dove i client vedono distribuzioni differenti dei dati.

Due estensioni sono diventate riferimenti standard per gestire il Non-IID. FedProx [4] aggiunge un termine prossimale alla loss locale che limita quanto ciascun client può allontanarsi dai pesi globali durante il training locale, mitigando il fenomeno del *client drift*. SCAFFOLD [5] utilizza invece *control variates* scambiate tra client e server per correggere il gradiente locale e ridurre la varianza dell'aggiornamento.

Entrambe queste tecniche operano *al livello del training locale* lato client: modificano la loss o il gradiente prima dell'aggregazione. Il meccanismo NLP-Weighted proposto nel presente lavoro opera invece *al livello dell'aggregazione* lato server, modulando i pesi con cui gli aggiornamenti dei client sono combinati. Le due categorie di tecniche sono complementari e in linea di principio componibili, come discusso in sezione 8.1.

Anche il livello dell'aggregazione è però stato oggetto di proposte esplicite, ed è qui che si colloca il predecessore concettualmente più vicino al presente lavoro. Anelli et al. [6] estendono FedAvg con una pesatura *multi-criterio*, in cui una funzione punteggio combina più criteri secondo priorità gerarchiche fissate dal progettista del sistema. I criteri effettivamente implementati e valutati — dimensione del dataset locale, diversità delle etichette, divergenza del modello locale rispetto al globale — sono tuttavia tutti *client-side*: derivano da informazioni che i client calcolano sui propri dati o sui propri aggiornamenti. Gli autori indicano l'impiego di criteri di dominio come sviluppo possibile, senza implementarlo né valutarlo. Il meccanismo proposto in questo lavoro si colloca esattamente in quel punto: il criterio di priorità non è ricavato dal training, ma da un canale informativo esterno ad esso (sezione 3.4).

Un ampio ecosistema di software supporta oggi il FL sperimentale. Flower [7] è il framework utilizzato in questo lavoro; fornisce astrazioni per client, server, strategie di aggregazione e simulazione basata su Ray. La sua flessibilità nella definizione di strategie personalizzate è ciò che ha reso possibile l'implementazione di NLP-Weighted come Strategy custom senza modifiche al core del framework.

2.2 Road Damage Detection

La rilevazione automatica del degrado stradale a partire da immagini acquisite da veicoli in movimento è un problema di *object detection* che ha ricevuto attenzione crescente negli ultimi anni. Il dataset di riferimento del dominio è RDD2022 [1], raccolto e distribuito da Arya et

al. nell'ambito del CRDDC 2022 [2]. RDD2022 contiene circa 47 000 immagini con oltre 55 000 istanze annotate di quattro classi di danni stradali (fessure longitudinali, trasversali, ad alligatore e buche), provenienti da sei paesi con caratteristiche stradali significativamente diverse.

La maggior parte dei lavori sul RDD2022 utilizza approcci di training centralizzati con reti tipo YOLO [8] come architettura di base. I modelli sottomessi al CRDDC 2022 hanno raggiunto, sulla *leaderboard* relativa ai sei paesi combinati, valori di F1 compresi tra 0.60 e 0.77 per le prime undici soluzioni, con il miglior modello (un *ensemble* di reti YOLO e Faster R-CNN) a 0.77 [2].

Il lavoro più vicino al presente in termini di setup FL è Saha et al. [9], che applica il paradigma FedAvg a sottoinsiemi di RDD2022 usando YOLOv5l con Flower come framework. Gli esperimenti considerano sia uno scenario *single-country* (dataset del Giappone, JRDD) sia uno scenario *multi-country* (MRDD, con Giappone, India e Stati Uniti come client). Gli autori mostrano che i modelli federati raggiungono un mAP inferiore del 21–25% rispetto ai corrispondenti modelli centralizzati addestrati sull'unione dei dati, ma nel setup multi-country Non-IID superano di 1.33%–163% i modelli locali dei singoli paesi quando questi sono valutati sul test set globale. Un ulteriore risultato interessante riguarda l'applicazione del modello FL multi-country a un paese che non ha partecipato al training (Repubblica Ceca), su cui il modello FL ottiene performance comparabili al modello centralizzato, mostrando capacità di generalizzazione trasferibile.

Questo lavoro usa FedAvg senza modifiche e non introduce segnali esterni al training locale per modulare l'aggregazione. È quindi complementare al presente lavoro, che introduce esplicitamente un canale contestuale (NLP sulle lamentele cittadine) per modulare i pesi.

2.3 NLP e civic reporting per smart city

L'analisi di segnalazioni testuali dei cittadini nell'ambito di piattaforme di *civic reporting* (di cui il NYC 311 è l'esempio più noto e studiato) ha attirato interesse crescente nella letteratura di *smart city* e *urban informatics*.

Kontokosta et al. [10] hanno mostrato che i volumi di segnalazioni al NYC 311 non sono uniformi rispetto alla distribuzione oggettiva dei problemi urbani sottostanti: fattori socio-demografici (reddito, densità abitativa, proprietà dell'immobile) influenzano significativamente la propensione dei cittadini a segnalare. Un'osservazione analoga è riportata da Wang et al. [11], che hanno usato la struttura delle segnalazioni al 311 come "signature" del contesto urbano locale, identificando cluster di aree cittadine con profili di segnalazione distinti.

Questi lavori documentano un aspetto rilevante per il presente progetto: la correlazione tra segnalazioni testuali e distribuzione reale dei problemi è imperfetta e sistematicamente distorta da fattori esterni al problema stesso. La scelta di costruire il dataset di lamentele sintetiche del presente lavoro (sezione 4.6) con distribuzione correlata ma non identica al vettore Oracle riflette esplicitamente questa osservazione empirica.

In generale, la letteratura di *urban informatics* riconosce le piattaforme di *civic reporting* come fonte di dati testuali geograficamente localizzati, potenzialmente utili come *input* a modelli

predittivi di dominio ma soggetti a distorsioni sistematiche dovute a fattori socio-demografici, come mostrato dai lavori citati sopra.

Categorizzazione degli approcci alla Sentiment Analysis. La letteratura sulla *sentiment analysis* si organizza in quattro famiglie. Gli approcci *lexicon-based* (o *knowledge-based*) assegnano a ciascun termine un punteggio di polarità attinto da un *opinion lexicon* e aggregano i punteggi in un valore di documento; il lessico può essere costruito per annotazione manuale, per espansione da dizionario (VADER [12], SentiWordNet [13]) o per induzione da corpus a partire da *seed word*. Non richiedono dati di addestramento, ma dipendono dal dominio, poiché una stessa parola cambia connotazione al cambiare del contesto applicativo. Gli approcci di *machine learning* addestrano invece un classificatore su dati etichettati e si articolano in supervisionati — lineari (SVM, reti neurali), probabilistici (Naive Bayes, reti bayesiane), *deep learning* (CNN, RNN, Transformer) e ad albero di decisione — semi-supervisionati e non supervisionati; apprendono pattern specifici del dominio e ottengono in genere prestazioni superiori, al prezzo di un fabbisogno di dati annotati che cresce con la capacità del modello. Gli approcci *ibridi* combinano le due famiglie precedenti per unire la stabilità dei lessici all'accuratezza dei classificatori, impiegando i punteggi lessicali come *feature* del classificatore oppure componendone gli *output* in uno schema di tipo *ensemble*. Una quarta famiglia residuale raccoglie infine gli approcci che si distinguono per regime di addestramento o granularità dell'analisi, fra cui il *transfer learning*, l'*aspect-based* e la *multimodal sentiment analysis*.

Variabili che governano la scelta dell'approccio. L'impiego relativo delle quattro famiglie varia lungo due assi. Il primo è *temporale*: i metodi lexicon-based hanno dominato la fase iniziale del campo, in assenza di corpora annotati di taglia adeguata; il machine learning supervisionato su rappresentazioni sparse (TF-IDF, *bag-of-words*) li ha progressivamente sostituiti; l'introduzione dei Transformer pre-addestrati [14] ha infine reso dominante il *transfer learning* da modelli di linguaggio, riducendo il fabbisogno di annotazioni a valle. Il secondo asse è il *dominio*, e agisce sulle due famiglie principali in modo simmetrico: per i metodi lexicon-based determina la validità del lessico, per i metodi neurali determina quale corpus di pre-addestramento rende il modello adeguato al testo da analizzare. È su questo secondo asse che si colloca la scelta discussa di seguito.

Rispetto alla categorizzazione appena introdotta, il modulo di *sentiment*/severity del presente lavoro è un approccio **ibrido**: compone in modo additivo l'uscita di un classificatore neurale con euristiche lessicali di dominio (sezione 3.3.3). La componente neurale è Twitter-RoBERTa [15], un modello Transformer pre-addestrato su 124 milioni di tweet e distribuito nell'ambito del benchmark TweetEval [16], impiegato *off-the-shelf* e dunque in regime di *transfer learning* senza *fine-tuning*. La scelta di un modello pre-addestrato su tweet, anziché su recensioni di prodotti (`nlptown/bert-base-multilingual-uncased-sentiment`) o su corpus generici, agisce sulla variabile di dominio discussa sopra e riflette la maggiore vicinanza stilistica dei tweet alle lamentele cittadine (frasi brevi-medie, tono emotivo, contenuto puntuale). Il confronto empirico riportato in sezione 3.3.3 oppone la configurazione finale a un'alternativa che varia simultaneamente il modello e il peso del termine neurale: documenta dunque il divario fra le due configurazioni nel

loro complesso, non il contributo isolato della scelta del corpus di pre-addestramento. Il modulo di NER geografico usa spaCy con il modello standard `en_core_web_lg` [17], uno strumento consolidato nell'ecosistema NLP Python.

2.4 Posizionamento del contributo

Il contributo del presente lavoro si differenzia dai lavori discussi sopra su tre aspetti principali.

Innanzitutto, rispetto ai lavori sul FL per *road damage detection* [9, 18], introduciamo esplicitamente un canale contestuale esterno al training visuale. Il segnale NLP prodotto dalle lamentele cittadine non è usato come input al modello di object detection, ma come segnale di priorità per modulare l'aggregazione federata. Questa è un'estensione architetturale, non una modifica del rilevatore.

In secondo luogo, rispetto alla letteratura FL classica sul Non-IID [4, 5], il meccanismo proposto opera a un livello diverso (aggregazione lato server anziché training locale) e con una motivazione diversa (concentrare la capacità del modello su una classe di interesse identificata da un canale esterno, anziché correggere il *client drift* generico).

Infine, rispetto alla letteratura di *smart city* che analizza le segnalazioni civiche dei cittadini [10, 11, 19], in particolare i dataset 311 nordamericani, il presente lavoro utilizza il segnale NLP non come oggetto di studio ma come *input* a un sistema di training distribuito. Le lamentele non sono classificate per se stesse, ma tradotte in un vettore di urgenza che influenza il modo in cui il modello di rilevazione viene addestrato.

Il lavoro più vicino al presente per dominio applicativo è RoadFed [20], sistema FL multimodale per il rilevamento di pericoli stradali che integra immagini e testo. La differenza rispetto alla presente proposta è il punto di inserzione del segnale testuale: in RoadFed il testo è una modalità di *input* del rilevatore, fusa a livello di feature tramite estrattori paralleli, mentre l'aggregazione federata resta proporzionale alla dimensione dei dataset locali secondo lo schema FedAvg. Nel presente lavoro il testo non entra nel modello di visione: agisce esclusivamente sui pesi di aggregazione, lasciando invariata l'architettura del rilevatore.

Nel dominio del monitoraggio infrastrutturale il testo è stato impiegato anche come segnale ausiliario a livello di feature [21], dove però si tratta di metadati categoriali d'ispezione codificati in vettori one-hot, non di linguaggio naturale prodotto spontaneamente dai cittadini.

Alla data del presente lavoro non risultano proposte che combinino esplicitamente le due componenti: FL su object detection e canale contestuale NLP *esogeno* rispetto ai dati di training. I lavori più vicini seguono due strade distinte: impiegano il testo come modalità di input del modello [20, 21], lasciando l'aggregazione proporzionale alla dimensione dei dataset locali, oppure adattano i pesi di aggregazione a partire da segnali interni al training (gradienti, update dei parametri, performance locale) senza ricorrere ad alcun canale informativo esterno [6]. La combinazione dei due aspetti resta il contributo caratteristico del presente progetto. Alla valutazione empirica di questa combinazione è dedicato un protocollo sperimentale progettato per evitare la circolarità tra la definizione della criticità e il segnale che dovrebbe rilevarla (sezione 5).

3 Metodo

Questa sezione descrive l'architettura del sistema proposto e la sua implementazione. Il sistema è composto da due moduli principali: un modulo di *Federated Learning* per l'addestramento distribuito del rilevatore di danni stradali (sezione 3.2), e un modulo NLP che estrae un segnale di urgenza da lamentele testuali dei cittadini (sezione 3.3). I due moduli comunicano attraverso una singola interfaccia astratta comune (sezione 3.4), che consente di sostituire l'implementazione del produttore del segnale di urgenza senza modifiche al consumatore. Questa modularità è sfruttata sia per la validazione controllata del sistema (Sezione 5) sia per la dimostrazione *end-to-end* con la pipeline NLP reale (sezione 6.4).

3.1 Architettura ad alto livello

L'idea centrale del sistema è quella di combinare due canali di informazione complementari nel contesto del monitoraggio del degrado stradale urbano:

- Un **canale automatico e visuale**, in cui una flotta di veicoli distribuiti (nel nostro esperimento, associati ai sei paesi del dataset RDD2022) addestra localmente un rilevatore di danni stradali su immagini acquisite dalle proprie camere, sincronizzandosi periodicamente con un server federato secondo il paradigma FedAvg [3].
- Un **canale testuale e umano**, in cui i cittadini segnalano problemi stradali attraverso descrizioni in linguaggio naturale (una piattaforma di *civic reporting*). Da questi testi, una pipeline NLP estrae informazioni strutturate: la localizzazione geografica e un punteggio di urgenza proveniente dal tono e dal contenuto della segnalazione.

Il sistema utilizza il segnale prodotto dal canale testuale per modulare i pesi con cui il server federato aggrega gli aggiornamenti del canale visuale. L'intuizione è che i client operativi in aree segnalate come più critiche dai cittadini (molte lamentele, tono urgente) dovrebbero contribuire di più all'aggiornamento del modello globale sulla classe critica D40. Il canale testuale funge quindi da *oracolo contestuale*: fornisce al FL informazione che non può essere estratta dai soli dati visuali e che può rompere la simmetria imposta dalla pesatura per volume di dati tipica di FedAvg.

Figura 1 mostra lo schema logico del sistema. Le due componenti sono deliberatamente disaccoppiate: il modulo NLP produce un vettore di urgenza per client, e il modulo FL lo consuma nel proprio server di aggregazione. L'interfaccia comune `UrgencyProvider` formalizza questo disaccoppiamento.

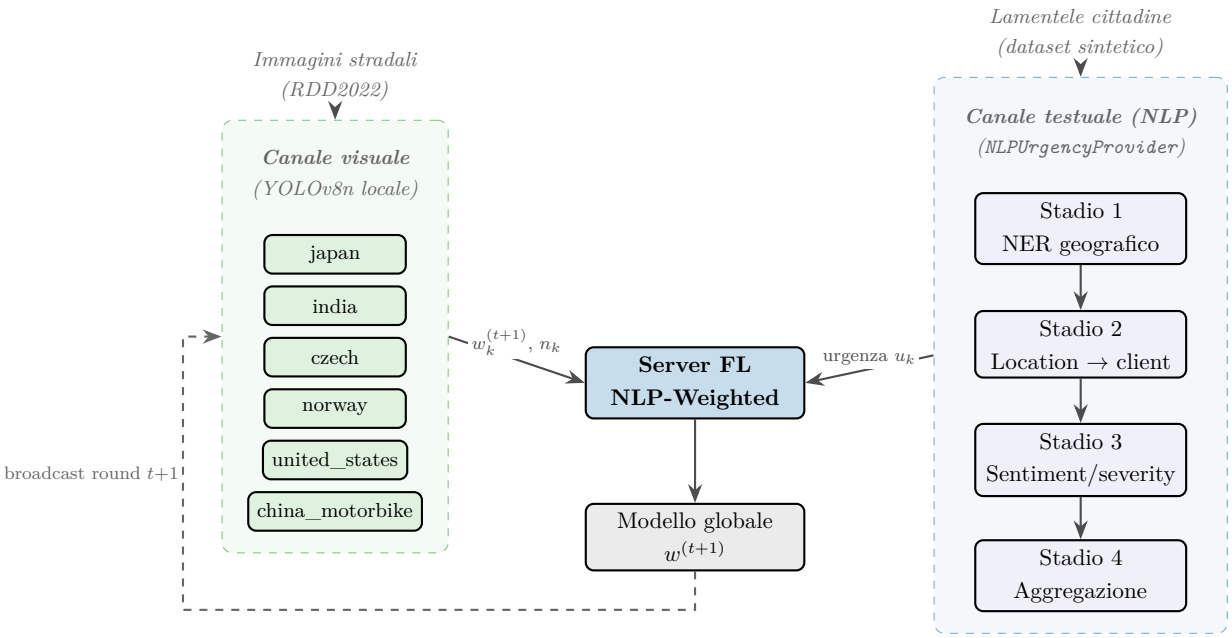


Figura 1: Architettura ad alto livello del sistema. Due canali di informazione (visuale a sinistra, testuale a destra) convergono nel server federato che esegue l'aggregazione NLP-WEIGHTED. Il canale visuale produce gli aggiornamenti locali dei pesi $w_k^{(t+1)}$ e il volume di dati n_k; il canale testuale produce, tramite la pipeline NLP a quattro stadi, il vettore di urgenza per client u_k. Il server combina i due contributi e re-invia il modello aggiornato $w^{(t+1)}$ ai client per il round successivo.

3.2 Modulo Federated Learning

3.2.1 Rilevatore di base

Il rilevatore di danni stradali usato in tutti gli esperimenti è YOLOv8n, la variante *nano* della famiglia YOLOv8 [8]. Con circa 3.2 milioni di parametri, è il modello più leggero della famiglia ed è adatto a scenari *edge-first* come quello considerato, in cui i client federati sono dispositivi con budget di calcolo limitato (veicoli in movimento). Il modello è inizializzato dai pesi pre-addestrati su COCO, con la testa di rilevamento rimappata a quattro classi (D00, D10, D20, D40) tramite ricostruzione del `DetectionModel` con `nc=4` e trasferimento dei pesi condivisibili con lo stato pre-addestrato.

3.2.2 Protocollo federato

Ogni round federato t segue il classico schema di FedAvg:

1. Il server invia i pesi globali correnti $w^{(t)}$ a tutti i client partecipanti.
2. Ciascun client k esegue `local_epochs` epoche di discesa del gradiente su una copia locale del modello, producendo pesi aggiornati $w_k^{(t+1)}$ e comunicando al server anche il numero di esempi di training n_k.
3. Il server aggrega gli aggiornamenti tramite media pesata:

$$w^{(t+1)} = \sum_{k=1}^{K} \pi_k^{(t)} \cdot w_k^{(t+1)}, \qquad \sum_{k=1}^{K} \pi_k^{(t)} = 1$$

4. Il server esegue una fase di valutazione (*evaluate*) distribuita, in cui ogni client valuta il modello aggregato sul proprio insieme di validazione; le metriche (mAP@50, mAP@50-95, critical_ap50) sono aggregate come media pesata sul numero di esempi di validazione.

La differenza tra le strategie considerate in questo lavoro sta esclusivamente nel modo in cui i pesi di aggregazione $\pi_k^{(t)}$ sono calcolati.

3.2.3 FedAvg (*baseline context-blind*)

In FedAvg [3], i pesi sono proporzionali al volume di dati locali:

$$\pi_k^{(t)} = \frac{n_k}{\sum_{j=1}^{K} n_j}.$$

È la scelta più naturale e la baseline standard di ogni sistema FL. Il suo limite in scenari Non-IID come il nostro è che ignora completamente il contenuto qualitativo dei dati: un client con molti dati ma poco informativi sulla classe critica riceve più peso di uno con pochi dati ma molto informativi.

3.2.4 Oracle-Weighted

Come primo passo verso una pesatura contestuale, definiamo la strategia ORACLE-WEIGHTED, in cui i pesi combinano la componente volume-based di FedAvg con una componente basata su un segnale di *criticità* $c_k \in [0, 1]$ (con $\sum_k c_k = 1$) noto al server:

$$\pi_k^{(t)} = \alpha \cdot \frac{n_k}{\sum_j n_j} + (1 - \alpha) \cdot c_k, \qquad \alpha \in [0, 1].$$

Il parametro α regola il compromesso tra pesatura per volume (standard) e pesatura per criticità. Con $\alpha = 1$ si recupera esattamente FedAvg; con $\alpha = 0$ i pesi seguono interamente il segnale di criticità. In tutti gli esperimenti abbiamo usato $\alpha = 0.5$ come default equilibrato.

Nel nostro protocollo (sezione 5), c_k è definito come la quota della classe critica D40 presente nelle annotazioni di training e validazione del client k, normalizzata:

$$c_k = \frac{\text{numero istanze di D40 in dataset}_k}{\sum_j \text{numero istanze di D40 in dataset}_j}.$$

Questa è informazione statica, calcolata una sola volta dal manifest del dataset. La strategia ORACLE-WEIGHTED rappresenta quindi un *upper bound informativo* rispetto alle strategie che devono inferire il segnale di criticità da un canale esterno (come NLP-WEIGHTED, sezione 3.2.5).

3.2.5 NLP-Weighted

La strategia principale del progetto, NLP-WEIGHTED, generalizza ORACLE-WEIGHTED rendendo il segnale di criticità dinamico e proveniente da una sorgente esterna (nel deployment target, la pipeline NLP sulle lamentele cittadine). Il segnale non è più noto a priori come c_k statico, ma è prodotto a ogni round t da un componente astratto:

Listing 1: Interfaccia `UrgencyProvider`.

```python
class UrgencyProvider(ABC):
    """Sorgente del segnale di urgenza per client e per round."""
    @abstractmethod
    def get_urgency(
        self,
        round_idx: int,
        client_names: list[str],
    ) -> dict[str, float]:
        """Restituisce {client_name: urgency_score}."""
        ...
```

I pesi di aggregazione sono quindi calcolati come:

$$\pi_k^{(t)} = \alpha \cdot \frac{n_k}{\sum_j n_j} + (1 - \alpha) \cdot \tilde{u}_k^{(t)},$$

dove $\tilde{u}_k^{(t)}$ è ottenuto normalizzando l'output del provider e applicando due salvaguardie descritte di seguito.

Salvaguardie: cap e time decay. Un problema noto delle strategie di aggregazione fortemente sbilanciate verso pochi client è il rischio di *catastrophic forgetting*: se il server dà per più round di seguito un peso molto alto a un singolo client, il modello globale sovra-adatta al suo dataset locale e perde performance sui client sotto-pesati. Per mitigare questo rischio abbiamo introdotto due salvaguardie:

- **Cap** $c \in (0, 1]$: nessun singolo client può ricevere un peso maggiore di c. I pesi che superano il cap vengono ridimensionati e la massa in eccesso redistribuita proporzionalmente agli altri client. Nel default $c = 0.5$: nessun client può dominare per più della metà del peso complessivo.
- **Time decay** $\gamma \in (0, 1]$: attenua progressivamente l'influenza del segnale di urgenza all'aumentare del round, come se il segnale "invecchiasse". Detto $u_k^{(t)}$ l'output normalizzato del provider al round t, il contributo di urgenza effettivamente usato è

  $$\tilde{u}_k^{(t)} = \frac{1}{K} + \gamma^{\,t-1} \left(u_k^{(t)} - \frac{1}{K} \right),$$

  che per $\gamma < 1$ tende alla distribuzione uniforme sui client al crescere di t; l'esponente $t - 1$ lascia il primo round non attenuato. Nel default $\gamma = 1$ il fattore $\gamma^{\,t-1}$ vale 1 a ogni round, la formula si riduce a $\tilde{u}_k^{(t)} = u_k^{(t)}$ e il decadimento è dunque inattivo.

Nella sensitivity analysis (sezione 6.3) esploriamo una griglia 3×3 di valori di questi due iperparametri.

3.2.6 Provider disponibili

Il pattern architetturale a interfaccia astratta consente di implementare diversi provider dello stesso segnale. Il progetto include tre implementazioni:

- `OracleUrgencyProvider`: restituisce a ogni round il vettore di criticità statico c_k calcolato dal manifest. Con il default $\gamma = 1$, in cui la formula del *time decay* si riduce a un'identità, NLP-WEIGHTED su questo provider calcola gli stessi pesi di ORACLE-WEIGHTED: le due strategie sono in quel caso algebricamente equivalenti, e la coincidenza funge da verifica di correttezza dell'interfaccia. È il provider impiegato come *upper bound* informativo.

- `NoisyOracleUrgencyProvider`: applica al segnale Oracle un rumore controllato. Ad ogni round, il vettore restituito è $\tilde{c}_k = a \cdot c_k + (1 - a) \cdot \sigma(c)_k$, dove $a \in [0, 1]$ simula l'accuratezza di un ipotetico modulo NLP e $\sigma(c)$ è una permutazione casuale di c (generata da un RNG seedato). Con $a = 1$ non c'è rumore; con $a = 0$ il segnale è completamente random. Questo provider è centrale al Componente C del protocollo di validazione (sezione 5), che caratterizza la robustezza del metodo al rumore.

- `NLPUrgencyProvider`: esegue la pipeline NLP reale (sezione 3.3) su un dataset testuale di lamentele. Usato per la dimostrazione *end-to-end* del sistema (sezione 6.4).

Va sottolineato un aspetto importante di questa architettura: *lo sweep sperimentale principale* (sezione 4.7, sezione 6.1, sezione 6.2) usa i due provider sintetici, non la pipeline NLP reale. La scelta è metodologica e motivata in dettaglio in sezione 5: i provider sintetici consentono di variare in modo controllato l'accuratezza del segnale, cosa impossibile con un modulo NLP reale il cui error rate è fisso. La pipeline NLP reale è valutata separatamente come dimostrazione funzionale.

3.3 Modulo Natural Language Processing

Il modulo NLP prende in input un insieme di lamentele testuali dei cittadini e produce in output un vettore di urgenza per client compatibile con l'interfaccia `UrgencyProvider`. La pipeline è strutturata in quattro stadi sequenziali, ciascuno responsabile di una singola trasformazione:

1. **Stadio 1 — NER geografico**: estrae le entità geografiche dal testo (sezione 3.3.1).
2. **Stadio 2 — Mapping località→client**: associa ogni lamentela a un client federato (sezione 3.3.2).
3. **Stadio 3 — Severity classification**: calcola uno score di gravità per ciascuna lamentela (sezione 3.3.3).
4. **Stadio 4 — Aggregazione per client**: combina le lamentele in un vettore di urgenza per client (sezione 3.3.4).

Ciascuno stadio è implementato come un modulo Python indipendente (package `src/nlp/`) e testato singolarmente attraverso *smoke test* integrati.

3.3.1 Stadio 1 — NER geografico

Il primo stadio individua nel testo della lamentela le entità geografiche che permettono di localizzare il problema segnalato. Si usa il modello pre-addestrato `en_core_web_lg` di spaCy [17], il cui componente NER è addestrato su OntoNotes 5, con NER in grado di riconoscere le categorie standard rilevanti:

- `GPE` (*Geopolitical Entity*): città, stati, nazioni (*es.* "Tokyo", "India");
- `LOC` (*Location*): località non-politiche (*es.* "Shibuya crossing");
- `FAC` (*Facility*): strade, edifici, aeroporti (*es.* "Main Street").

Il modulo restituisce, per ogni testo, la località "principale" selezionata con la priorità GPE > LOC > FAC. Se il testo menziona sia una via che una città, è la città a determinare il client target: la strada è localizzata rispetto a un'area geografica più ampia. Nella valutazione sul dataset sintetico (sezione 4.6), lo Stadio 1 estrae correttamente la località principale in 55/60 casi (91.7%).

Non viene effettuato alcun *fine-tuning* del modello NER. La scelta è motivata sia dall'esiguità del dataset annotato disponibile (60 esempi, insufficienti per un training statistico robusto), sia dal fatto che *fine-tuning* su piccoli dataset comporta un rischio significativo di *catastrophic forgetting* rispetto alle capacità del modello base. L'uso di un modello *off-the-shelf* pre-addestrato è la strategia standard per casi d'uso con poche annotazioni.

3.3.2 Stadio 2 — Mapping località→client

Il secondo stadio associa la località estratta dallo Stadio 1 a uno dei sei client federati. Poiché i client corrispondono a paesi (sezione 4.2), il mapping è a granularità paese: una lamentela su "Tokyo" viene assegnata al client `japan`, una su "Mumbai" al client `india`, e così via. La tabella di mapping è una struttura dati statica che elenca le principali città e regioni di ogni paese del CRDDC.

L'implementazione combina due strategie complementari:

- **Match semantico via NER**: se lo Stadio 1 ha estratto un'entità geografica, si tenta un *lookup* nella tabella (con *fuzzy matching* per gestire varianti come "Shibuya crossing").
- **Fallback *pattern-match* sul testo grezzo**: se la strategia precedente fallisce (perché il NER non ha trovato entità, o perché l'entità trovata non è nella tabella), si effettua una ricerca sottostringa *case-insensitive* delle città note direttamente nel testo. In caso di corrispondenze multiple, vince la più lunga (ad esempio "New York City" prima di "New York").

Questa strategia ibrida è una pratica consolidata nei sistemi di produzione NLP: il modello neurale ha alta precisione semantica ma copertura limitata sui nomi propri fuori distribuzione, mentre il *pattern-match* ha copertura elevata ma nessuna comprensione semantica. La combinazione dei due massimizza la robustezza. Sul dataset sintetico bilanciato, lo Stadio 2 raggiunge il 100%

di accuratezza, recuperando in particolare 5 lamentele che il solo NER non riusciva a mappare (città meno frequenti come Ostrava, Fredrikstad, Sandnes). Sul dataset realistico l'accuratezza è leggermente inferiore: il vettore di urgenza prodotto dalla pipeline (tabella 5) differisce lievemente da quello ricostruibile dalle etichette di riferimento, per effetto di alcune assegnazioni errate.

Generalizzabilità intra-città. La stessa architettura è utilizzabile senza modifiche in un deployment reale in cui i client corrispondano a veicoli o zone di pattugliamento all'interno di una singola città. In tal caso la tabella statica città→paese verrebbe sostituita da un servizio di *geocoding* (ad esempio OpenStreetMap Nominatim) e da una mappa dei poligoni delle zone di ciascun client. Gli altri stadi della pipeline non richiederebbero alcuna modifica.

3.3.3 Stadio 3 — Severity classification ibrida

Il terzo stadio quantifica la gravità di ciascuna lamentela come uno score continuo in $[0, 1]$. La sfida principale di questo stadio è il *gap* tra *sentiment* generico (positivo/negativo/neutro) e *severity* di dominio (bassa/media/alta urgenza infrastrutturale). Un modello di *sentiment* generico può essere insensibile a lamentele oggettivamente gravi ma espresse in tono formale ("Reporting life-threatening damage") e sovra-reagire a lamentele banali espresse in tono arrabbiato.

Per gestire questo *gap*, lo Stadio 3 combina un modello Transformer con euristiche lessicali di dominio.

Componente neurale. Si usa il modello pre-addestrato `cardiffnlp/twitter-roberta-base-sentiment-latest` [15], un RoBERTa addestrato su 124 milioni di tweet per la classificazione di *sentiment* in tre classi (*negative*, *neutral*, *positive*) con *confidence*. La scelta di un modello addestrato su tweet, anziché ad esempio su recensioni di prodotti, è motivata dalla maggiore vicinanza stilistica dei tweet alle lamentele cittadine (frasi brevi-medie, tono emotivo, contenuto puntuale). Il modello è usato *off-the-shelf*, senza *fine-tuning*, per le stesse ragioni discusse per lo Stadio 1. L'output del Transformer viene mappato in un valore di severity in $[0, 1]$: *negative* con confidence s produce $0.5 + 0.5s$; *positive* con confidence s produce $0.5 - 0.5s$; *neutral* produce 0.5.

Euristiche lessicali di dominio. Al segnale del Transformer si sommano tre *boost* euristici sensibili al vocabolario delle lamentele stradali:

- **Keyword *boost***: presenza di parole tipiche di alta urgenza (*pothole*, *dangerous*, *urgent*, *immediate*, *life-threatening*, *someone will die*, ecc.) o di bassa urgenza (*small*, *minor*, *log for reference*, *low priority*, ecc.), che spostano il punteggio verso l'alto o il basso.
- **Caps *boost***: parole in maiuscolo di almeno tre caratteri, indicative di enfasi ortografica.
- **Exclamation *boost***: presenza di uno o più punti esclamativi.

Un quarto meccanismo, il *medium pull*, attira lo score verso il centro (0.5) in presenza di indicatori espliciti di priorità media (*es.* "medium priority", "within the month").

Combinazione. Detto p il punteggio preliminare, il punteggio finale è:

$$p = w_T \cdot s_T + (1 - w_T) \cdot 0.5 + b_{\text{kw}} + b_{\text{caps}} + b_{\text{excl}}, \qquad s = \text{clip}_{[0,1]}\big((1 - m_{\text{pull}})\, p + m_{\text{pull}} \cdot 0.5\big).$$

Il *medium pull* non è un termine additivo ma una combinazione convessa verso il centro: con $m_{\text{pull}} = 0$ il punteggio resta invariato, con $m_{\text{pull}} = 1$ collasserebbe su 0.5.

Il coefficiente $w_T = 0.4$ non ripartisce il peso tra due componenti. Poiché $w_T s_T + (1 - w_T) \cdot 0.5 = 0.5 + w_T(s_T - 0.5)$, il suo effetto è contrarre verso il valore neutro lo scostamento prodotto dal Transformer, riducendolo al 40%. La componente neurale da sola è quindi confinata in $[0.3, 0.7]$, e i valori estremi di severity sono determinati interamente dai *boost* lessicali, che entrano in modo additivo e la cui influenza dipende dall'ampiezza dei rispettivi coefficienti. In questo senso il vocabolario di dominio pesa più del *sentiment* generico, cui resta un ruolo di modulazione attorno al neutro.

La configurazione è stata scelta dopo il confronto empirico con un'alternativa che usava $w_T = 1.0$ e il modello `nlptown/bert-base-multilingual-uncased-sentiment`, addestrato su recensioni di prodotti, che ha mostrato saturazione verso "alta severity" su vocabolario tecnico e sotto-classificazione delle lamentele formali gravi (63.3% di *accuracy* nel confronto categorico), evidenziando il *gap sentiment–severity*. Il confronto varia però simultaneamente due fattori (il peso e il modello) e non consente di attribuire il divario all'uno o all'altro. La configurazione ibrida finale raggiunge l'86.7% di *accuracy* sul dataset bilanciato di valutazione (tabella 17) e l'83.3% su quello realistico.

3.3.4 Stadio 4 — Aggregazione per client

Il quarto stadio combina l'insieme delle lamentele processate in un vettore di urgenza per client. Sia $\mathcal{L}$ l'insieme delle lamentele valide (con client assegnato dallo Stadio 2 e severity computata dallo Stadio 3), e sia s_ℓ la severity della lamentela $\ell \in \mathcal{L}$. Definiamo l'*urgenza grezza* del client k come la somma delle severity delle sue lamentele:

$$u_k^{\text{raw}} = \sum_{\ell \in \mathcal{L}_k} s_\ell, \qquad \mathcal{L}_k = \{\ell \in \mathcal{L} \mid \text{client}(\ell) = k\}.$$

Questa formulazione scala naturalmente sia con il volume di lamentele che con la loro intensità media (le due letture sono algebricamente equivalenti: $u_k^{\text{raw}} = |\mathcal{L}_k| \cdot \bar{s}_k$).

Per evitare che client senza lamentele ricevano peso zero (con conseguente esclusione totale dall'aggregazione FL), si applica una soglia minima $u_{\min} = 0.01$:

$$u_k^{\text{clipped}} = \max(u_k^{\text{raw}}, u_{\min}).$$

Il pattern è comune nei sistemi reali: assenza di segnalazioni non implica assenza di problemi, ed è prudente non azzerare il contributo di un client sulla sola base del silenzio.

Il vettore finale è ottenuto normalizzando a somma 1, in modo da essere direttamente compatibile

con l'interfaccia `UrgencyProvider`:

$$u_k = \frac{u_k^{\text{clipped}}}{\sum_j u_j^{\text{clipped}}}.$$

3.4 Integrazione dei due moduli

L'integrazione dei moduli FL e NLP passa esclusivamente attraverso l'interfaccia `UrgencyProvider`. La classe `NLPUrgencyProvider` è un semplice *wrapper* che alla sua istanziazione carica il dataset di lamentele, esegue in sequenza i quattro stadi della pipeline NLP, e memorizza il vettore di urgenza risultante. Il metodo `get_urgency(round_idx, client_names)` restituisce ad ogni chiamata questo vettore precomputato, coerentemente con l'ipotesi che il segnale NLP sia raccolto in background dal sistema e sia statico rispetto al round di training.[2]

Dal punto di vista del modulo FL, `NLPUrgencyProvider` è indistinguibile da `OracleUrgencyProvider` o `NoisyOracleUrgencyProvider`: implementa la stessa interfaccia, ritorna un `dict[str, float]` con la stessa semantica, e non richiede alcuna modifica al codice di aggregazione. Questa è la proprietà chiave che consente di riservare lo sweep sperimentale principale ai provider sintetici (necessari per il controllo del rumore, sezione 5) e di dimostrare separatamente la funzionalità end-to-end della pipeline NLP reale (sezione 6.4), senza mai duplicare il codice di integrazione.

4 Setup sperimentale

Questa sezione descrive il contesto empirico su cui è stato costruito e valutato il sistema: il dataset di partenza, il partizionamento *Non-IID* adottato per simulare la distribuzione dei dati fra client federati, il framework tecnologico, le scelte di implementazione, le metriche di valutazione e la scala complessiva della campagna sperimentale. Il protocollo di validazione che dà senso metodologico a queste scelte è discusso separatamente in sezione 5.

4.1 Dataset: RDD2022

Il progetto utilizza RDD2022 [1], il dataset ufficiale del *Crowdsensing-based Road Damage Detection Challenge* (CRDDC) [2], distribuito su figshare con DOI `10.6084/m9.figshare.21431547`. Il dataset raccoglie circa 47 000 immagini di strade acquisite da veicoli in sei paesi diversi: Repubblica Ceca, India, Giappone, Norvegia, Stati Uniti d'America e Cina (sottoinsieme *motorbike*). Le annotazioni sono nel formato Pascal VOC (XML) e sono state convertite nel formato YOLO tramite lo script `scripts/convert_voc_to_yolo_all.py` prima dell'addestramento.

Il challenge originale definisce quattro classi di danno standard, comuni a tutti i paesi:

- **D00**: fessura longitudinale (*longitudinal crack*);
- **D10**: fessura trasversale (*transverse crack*);
- **D20**: fessura ad alligatore (*alligator crack*);

[2] Per un deployment con lamentele in *streaming* l'implementazione dovrebbe essere estesa con un metodo per aggiungere nuove lamentele fra round e ricalcolare l'urgenza incrementalmente. Si veda la sezione 8.

- **D40**: buca (*pothole*).

Alcuni sottoinsiemi del dataset contengono ulteriori classi paese-specifiche (D43, D44, D50), presenti quasi esclusivamente nelle annotazioni giapponesi. Coerentemente con la letteratura sul CRDDC [2], tali classi sono state escluse dal training e dalla valutazione, per garantire un vocabolario di classi uniforme fra i client. Il codice di conversione documenta esplicitamente lo scarto delle classi non standard e ne riporta i conteggi per trasparenza.

Tabella 1 riporta la distribuzione finale di immagini di training e validazione per paese, ottenuta dopo la conversione XML→YOLO. La colonna D40 è evidenziata perché questa classe sarà definita come *classe critica esogena* nel protocollo di validazione (sezione 5).

Tabella 1: Distribuzione del dataset RDD2022 dopo la conversione al formato YOLO. Le colonne Train e Val riportano il numero di immagini; le colonne D00–D40 il numero di annotazioni per classe sui due split combinati (una stessa immagine può contenere più istanze, anche di classi diverse). La colonna D40 corrisponde alla classe critica *pothole*.

Paese	**Train**	**Val**	**D00**	**D10**	**D20**	**D40**
Repubblica Ceca (`czech`)	2 405	424	988	399	161	197
India (`india`)	6 551	1 155	1 555	68	2 021	3 187
Giappone (`japan`)	8 931	1 575	4 049	3 979	6 198	2 243
Norvegia (`norway`)	6 937	1 224	8 570	1 730	468	461
Stati Uniti (`united_states`)	4 085	720	6 750	3 295	834	135
Cina motorbike (`china_motorbike`)	1 681	296	2 678	1 096	641	235
Totale	**30 590**	**5 394**	**24 590**	**10 567**	**10 323**	**6 458**

4.2 Partizionamento Non-IID: client come paesi

Il sistema è stato progettato assegnando a ciascun paese del CRDDC un distinto client federato, per un totale di sei client. Questa scelta è naturale rispetto alla struttura del dataset RDD2022 (che è già partizionato per paese) e produce un partizionamento *Non-IID* severo, la condizione più interessante dal punto di vista metodologico per valutare strategie di aggregazione contestuali.

L'eterogeneità è visibile immediatamente in tabella 1: la classe D40 è concentrata quasi esclusivamente in India (49% delle annotazioni totali di D40) e Giappone (35%), mentre negli Stati Uniti conta solo 135 istanze (2% del totale). Simmetricamente, la classe D00 è dominata da Norvegia (35%) e Stati Uniti (27%). Questa disparità produce un segnale forte per il meccanismo di aggregazione: un metodo *context-blind* come FedAvg tenderà a mediare gli aggiornamenti locali penalizzando i client con dataset piccoli (Repubblica Ceca, Cina), indipendentemente dal fatto che essi possano contribuire con informazione utile alla classe critica.

Considerazioni sulla scala geografica. Il proposal originale del progetto prevedeva un deployment in cui i client corrispondevano a veicoli o zone di pattugliamento all'interno di una singola città. Nell'implementazione, per vincoli sul dataset disponibile (non esistono, a nostra conoscenza, dataset pubblici di *object detection* stradale con partizionamento intra-città a

granularità adeguata), i client corrispondono ai sei paesi del challenge. Questa scelta modifica la scala geografica ma preserva le proprietà metodologicamente rilevanti dell'esperimento:

1. eterogeneità Non-IID tra i client, con distribuzione fortemente asimmetrica della classe critica D40;
2. presenza di un segnale esterno di criticità (implementato in questo lavoro tramite `UrgencyProvider` sintetici, si veda sezione 5);
3. meccanismo di aggregazione pesata parametrizzabile.

Il fenomeno oggetto di studio — la relazione tra qualità del segnale NLP e performance FL sulla classe critica — è invariante rispetto alla granularità geografica dei client. Un deployment intra-città manterrebbe le stesse dinamiche a scala più fine, richiedendo la sostituzione della sola tabella di mapping località→client (sezione 3). Si torna su questo punto in sezione 8.

4.3 Framework tecnologico

Il sistema è stato realizzato in Python 3.10, utilizzando i seguenti componenti principali:

- **Deep Learning & Object Detection**: Ultralytics [8] versione 8.4 come framework per l'addestramento di YOLOv8n (il modello più leggero della famiglia, circa 3.2 milioni di parametri).
- **Federated Learning**: Flower [7] versione 1.29, con esecuzione in modalità *simulation* basata su Ray. Ogni round federato istanzia i sei client come attori Ray paralleli, sfruttando la GPU tramite `client_resources = {"num_gpus": 0.16}`: la frazione è una quota logica di scheduling, che consente a Ray di collocare i sei attori concorrentemente sulla stessa GPU, e non una partizione della memoria video, che resta condivisa (sezione 4.4).
- **Natural Language Processing**: spaCy [17] versione 3.x con il modello pre-addestrato `en_core_web_lg` per il *Named Entity Recognition*, e la libreria HuggingFace Transformers per il modulo di sentiment analysis (modello `cardiffnlp/twitter-roberta-base-sentiment-latest` [15]).
- **Analisi dati e figure**: NumPy, Pandas, Matplotlib.

L'intera pipeline è stata sviluppata con l'obiettivo esplicito di mantenere le tre componenti (FL, NLP, orchestrazione) modulari e disaccoppiate. In particolare, il modulo NLP espone i suoi risultati al modulo FL tramite un'interfaccia astratta comune (`UrgencyProvider`, si veda sezione 3), consentendo di sostituire un provider sintetico (`OracleUrgencyProvider`, `NoisyOracleUrgencyProvider`) con la pipeline NLP reale (`NLPUrgencyProvider`) senza modifiche al codice FL.

4.4 Hardware ed esecuzione

L'intera campagna sperimentale finale è stata eseguita su una macchina cloud dotata di GPU NVIDIA RTX A4000 (16 GB di VRAM), CPU AMD Threadripper Pro 5955WX (16 core)

e 128 GB di RAM di sistema, con sistema operativo Ubuntu 22.04 in ambiente WSL2 su host Windows 11. L'allocazione frazionale della GPU (`num_gpus=0.16`) consente di eseguire in parallelo tutti e sei i client di ogni round federato, con un utilizzo di picco della GPU intorno al 96–99% e un consumo di memoria di ~ 6.5 GB su 16 disponibili.

Il tempo di calcolo per una singola *run* di 10 round federati con 6 client, `imgsz=416` e `local_epochs=1` è di circa 30 minuti sull'hardware descritto. La campagna sperimentale completa (51 run) ha richiesto approssimativamente 25 ore di calcolo continuato.

4.5 Metriche di valutazione

La valutazione del sistema utilizza tre metriche complementari, tutte calcolate su ciascun client sul suo insieme di validazione e poi aggregate dal server federato come media pesata sul numero di esempi per client:

- mAP@50: *mean Average Precision* calcolata su tutte le classi con soglia di IoU pari a 0.5. È la metrica standard di object detection ed è usata come misura delle performance globali del sistema.
- mAP@50-95: mAP@50 calcolata a soglie di IoU da 0.5 a 0.95 con passo 0.05 e poi mediata. Misura più restrittiva della qualità delle bounding box.
- critical_ap50: la *Average Precision* a IoU 0.5 calcolata esclusivamente sulla classe critica D40. Questa è la **metrica primaria** del progetto, perché riflette direttamente l'obiettivo del meccanismo NLP-Weighted: migliorare le performance sulla classe critica senza necessariamente migliorare tutto.

Perché AP e non recall. Una scelta importante riguarda l'uso di AP@0.5 anziché recall come metrica primaria sulla classe critica. Ultralytics espone per default un valore di recall calcolato alla soglia di confidence che massimizza F1 sul batch corrente. Questa soglia varia da *run* a *run* e introduce una variabilità spuria che maschera il vero segnale sperimentale. AP@0.5, calcolata come area sotto la curva precision-recall, è invece indipendente dalla soglia e produce misure di gran lunga più stabili tra *run* indipendenti. Il valore di recall grezzo è comunque salvato nei risultati per fini diagnostici, ma non compare nelle tabelle o nelle figure finali.

4.6 Dataset testuale di lamentele sintetiche

Il modulo NLP (sezione 3.3) opera su un secondo dataset, distinto da RDD2022, contenente lamentele testuali di cittadini che descrivono problemi della sede stradale. Poiché non esiste un dataset pubblico di segnalazioni cittadine allineate al monitoraggio del degrado stradale e partizionabili sui client di RDD2022, il dataset è stato costruito sinteticamente.

Le lamentele sono state generate con l'ausilio di un modello linguistico di grandi dimensioni, a partire da un insieme di criteri fissati a priori: variazione del registro (dal formale al colloquiale), presenza di riferimenti geografici sia espliciti sia indiretti, e inclusione di casi ambigui sul piano della severity. Ogni lamentela è stata successivamente revisionata e annotata manualmente dagli

autori, che hanno scartato le formulazioni ripetitive e assegnato le etichette di client e severity in modo indipendente dal testo suggerito in fase di generazione. Va segnalato che questa procedura introduce un rischio metodologico specifico: il testo prodotto da un modello generativo può presentare regolarità stilistiche assenti nel linguaggio spontaneo dei cittadini, sulle quali il modulo NLP potrebbe appoggiarsi. La valutazione riportata in sezione 6 va quindi letta come prova di funzionamento della pipeline, non come stima della sua accuratezza in un deployment reale; il punto è ripreso in sezione 8.

Ciascuna lamentela è composta da un testo in lingua inglese — scelta vincolata dal modello NER adottato (`en_core_web_lg`, sezione 3.3) — e da due annotazioni di riferimento: il client di appartenenza, usato come *ground truth* per la valutazione degli Stadi 1–2, e un livello di severity in $\{\text{low}, \text{medium}, \text{high}\}$, usato come *ground truth* per lo Stadio 3. Le annotazioni sono state assegnate in fase di costruzione, prima di qualsiasi esecuzione della pipeline, in modo da evitare che la definizione delle etichette fosse influenzata dagli errori osservati sul sistema.

Il progetto include due varianti del dataset, destinate a scopi distinti. La variante *bilanciata* (`synthetic_complaints.csv`) contiene 60 lamentele ripartite in modo uniforme sui sei client, 10 ciascuno, con tutti e tre i livelli di severity rappresentati per ciascun client. Questa uniformità serve alla validazione del modulo NLP in isolamento: una distribuzione sbilanciata renderebbe le accuratezze per stadio non confrontabili tra client e gonfierebbe le metriche aggregate a favore delle classi maggioritarie. La variante *realistica* (`synthetic_complaints_realistic.csv`) contiene anch'essa 60 lamentele, ma distribuite tra i client in modo proporzionale alla presenza della classe critica D40 in RDD2022; è la variante impiegata nella *run* end-to-end del sistema completo (sezione 6.4).

La distribuzione della variante realistica è deliberatamente *correlata* ma non *identica* al vettore Oracle c_k (sezione 3.2.4). Una corrispondenza esatta renderebbe il segnale NLP una copia rumorosa dell'oracolo e la validazione circolare: il modulo si limiterebbe a ricostruire per via testuale un'informazione già nota. Lo scostamento riflette inoltre un fatto del dominio applicativo, ossia che le segnalazioni cittadine sono un proxy imperfetto della distribuzione oggettiva delle anomalie, poiché fattori culturali, demografici e di accessibilità dei canali di segnalazione introducono un divario sistematico tra anomalie presenti e anomalie riportate. Le distribuzioni complete di entrambe le varianti sono riportate in sezione B.2.

4.7 Configurazione dello sweep

Lo sweep sperimentale finale è strutturato in tre fasi, ciascuna progettata per rispondere a una specifica *research question* (la motivazione metodologica è in sezione 5):

- **RQ1** (confronto strategie): 3 strategie $\times$ 3 seed $=$ 9 *run*. Le strategie sono FedAvg, Oracle-Weighted e NLP-Weighted (con `OracleUrgencyProvider`, ovvero segnale NLP simulato ideale).
- **RQ2** (robustezza al rumore NLP): 5 livelli di accuratezza $\times$ 3 seed $=$ 15 *run*. NLP-Weighted con `NoisyOracleUrgencyProvider` ad accuracy $\in \{1.0, 0.9, 0.8, 0.7, 0.5\}$.

- **Ablation** (sensitivity iperparametri): griglia 3×3 di *cap* e *time_decay* $\times$ 3 seed = 27 *run*. NLP-WEIGHTED con `OracleUrgencyProvider` e *cap* $\in \{0.3, 0.5, 0.7\}$, *time_decay* $\in \{0.85, 0.95, 1.00\}$.

Per un totale di **51 *run* federate** indipendenti. Ogni *run* è configurata con 10 round federati, un'epoca locale per round, dimensione delle immagini 416×416 pixel e *batch size* 16. Il seed globale (che controlla il *data loader*, il *sampler* di Ultralytics e il generatore di rumore del `NoisyOracleUrgencyProvider`) è parametrizzato e varia sui tre valori $\{42, 123, 2024\}$ per garantire riproducibilità e stimare la varianza da seed. L'elenco completo degli iperparametri (compresi α, *cap*, *time_decay* nella configurazione di default) è riportato in sezione A.

L'esecuzione dello sweep è orchestrata dallo script `scripts/run_full_pipeline.py`, che lancia sequenzialmente le 51 *run* tramite `subprocess` e ne conserva i *log* e i `results.json` in una gerarchia di cartelle per-fase/per-run. Un meccanismo di *resume* permette di riprendere lo sweep in caso di interruzione, saltando le *run* già completate. L'aggregazione dei risultati e la generazione delle figure e tabelle per il report sono automatizzate da `scripts/aggregate_pipeline_results.py`.

5 Protocollo di validazione

Il metodo proposto in sezione 3 pone un problema metodologico non banale: valutare l'efficacia di un meccanismo di aggregazione guidato da un segnale NLP significa dover definire cosa conta come "criticità" su cui misurare il miglioramento. La scelta di questa definizione condiziona tutto il resto della valutazione, e se non è curata può produrre un ragionamento circolare in cui il metodo appare superiore perché la metrica di riferimento coincide, per costruzione, con ciò che il metodo stesso ottimizza.

Questa sezione discute esplicitamente il problema di circolarità (sezione 5.1) e presenta il protocollo a tre componenti adottato per risolverlo: la definizione esogena della classe critica (sezione 5.2), l'*upper bound informativo* fornito dal provider Oracle (sezione 5.3), e la caratterizzazione controllata della robustezza al rumore NLP tramite il provider `NoisyOracleUrgencyProvider` (sezione 5.4). Il protocollo produce come sottoprodotto la struttura sperimentale a tre fasi (RQ1, RQ2, Ablation) già introdotta in sezione 4.7.

5.1 Il problema della circolarità

Il pericolo metodologico centrale del progetto è il seguente. Se il segnale NLP definisce quale classe è "critica" (ad esempio, concludendo che le lamentele riguardano prevalentemente le buche) e la valutazione di NLP-WEIGHTED è poi condotta misurando l'AP proprio su quella classe, si sta valutando il metodo con una metrica che il metodo stesso ha contribuito a scegliere. Il risultato sarebbe una tautologia: NLP-WEIGHTED appare superiore perché sta ottimizzando proprio ciò su cui viene misurato.

Un secondo pericolo, correlato, riguarda l'uso di provider di urgenza sintetici. Se `OracleUrgency-Provider` legge dal manifest la distribuzione della classe critica e NLP-WEIGHTED usa questo segnale per pesare l'aggregazione, la strategia è "allineata per costruzione" alla metrica sulla

classe critica. Il rischio, in questo caso, è di misurare non l'effetto del meccanismo di pesatura ma banalmente l'effetto di dare più peso ai client che hanno più istanze della classe che poi verrà valutata.

Entrambi i problemi si risolvono introducendo tre discipline metodologiche, che di seguito indichiamo come Componenti A, B, C del protocollo.

5.2 Componente A: definizione esogena della classe critica

La classe critica su cui valutare il sistema è definita **prima** di guardare qualsiasi output del segnale NLP o del sistema FL, sulla base di un criterio esterno e disciplinare al problema studiato. Nel presente lavoro, la classe critica D40 (*pothole*) è stata scelta sulla base della classificazione consolidata dei danni stradali nella letteratura di ingegneria delle infrastrutture: fra i tipi di danno considerati dal CRDDC, le buche sono riconosciute come danno strutturale prioritario, in quanto compromettono direttamente la sicurezza di circolazione dei veicoli e richiedono intervento rapido rispetto alle altre categorie di degrado (fessurazioni longitudinali, trasversali e ad alligatore).

Questa definizione esogena rompe la circolarità: nessuna scelta del segnale NLP, nessun risultato del sistema FL, e nessuna intuizione degli autori può modificare quale sia la classe critica. Se domani si volesse ripetere lo studio focalizzandosi su un'altra classe (ad esempio D20, *alligator cracking*, che indica un cedimento progressivo del manto stradale), sarebbe sufficiente sostituire lo standard di riferimento con una motivazione altrettanto esplicita.

La metrica primaria del progetto, critical_ap50, è definita di conseguenza come AP@0.5 calcolata esclusivamente su D40, in modo consistente con la Componente A.

5.3 Componente B: upper bound informativo con Oracle

Fissata la classe critica, si vuole poi separare il contributo di due fattori distinti:

- Il *massimo miglioramento raggiungibile* da una strategia di pesatura context-aware, quando il segnale di criticità è perfettamente informativo.
- Il *costo* pagato quando questo segnale non è perfetto, ma proviene da un modulo NLP realistico soggetto a errori.

Il primo fattore è quantificato dalla strategia Oracle-Weighted, o equivalentemente da NLP-Weighted con `OracleUrgencyProvider` e decadimento inattivo (sezione 3.2.4, sezione 3.2.6). Poiché Oracle legge direttamente dal manifest la distribuzione della classe critica — l'informazione che un ipotetico modulo NLP perfetto e privo di rumore riuscirebbe a inferire dalle lamentele — rappresenta un *upper bound informativo* rispetto a qualsiasi implementazione realistica di modulo NLP.

Il divario tra FedAvg e Oracle quantifica dunque il **beneficio massimo teorico** del meccanismo di aggregazione contestuale; il divario tra Oracle e una strategia con segnale NLP rumoroso quantifica il **costo** dell'imperfezione del segnale.

Questa scomposizione è utile in due direzioni. Da un lato, offre un tetto realistico all'aspettativa sul metodo: nessuna implementazione di modulo NLP potrà superare Oracle, per costruzione. Dall'altro, evita di attribuire alla strategia benefici che invece derivano già solo dall'atto di ridistribuire i pesi in modo Non-IID-aware.

5.4 Componente C: sweep controllato sul rumore NLP

Il terzo pilastro del protocollo caratterizza empiricamente la robustezza del sistema al rumore del segnale NLP. La domanda a cui si vuole rispondere è: fino a che punto può essere impreciso il segnale NLP, prima che il vantaggio di NLP-WEIGHTED rispetto a FEDAVG scompaia?

Rispondere a questa domanda con una pipeline NLP reale richiederebbe di produrre modelli con accuratezza a pari a valori arbitrari prefissati, cosa che nessuna reale procedura di *training* consente di garantire con precisione. Si sostituisce quindi la pipeline NLP con un provider sintetico (`NoisyOracleUrgencyProvider`, sezione 3.2.6) che prende in input il vettore Oracle e lo perturba con rumore controllato da un parametro $a \in [0, 1]$, interpretabile come accuratezza simulata del modulo NLP:

$$\tilde{c}_k^{(t)} = a \cdot c_k + (1 - a) \cdot \sigma^{(t)}(c)_k,$$

dove $\sigma^{(t)}(c)$ è una permutazione casuale del vettore c generata da un RNG seedato dal round t. Con $a = 1$ non c'è rumore (coincide con Oracle); con $a = 0$ il segnale è completamente random; i valori intermedi interpolano linearmente tra segnale utile e rumore.

Lo sweep di RQ2 esplora cinque valori di a $(1.0, 0.9, 0.8, 0.7, 0.5)$ su tre seed indipendenti, producendo una curva di degradazione della metrica primaria critical_ap50 in funzione dell'accuratezza NLP. La forma di questa curva risponde a due domande operative:

- Esiste un *plateau* di robustezza, cioè un intervallo di accuratezze in cui il metodo continua a funzionare come se il segnale fosse perfetto?
- Qual è il punto di *break-even* a cui NLP-WEIGHTED non offre più vantaggio rispetto a FEDAVG?

L'uso del provider sintetico è essenziale per rispondere a queste domande. Un modulo NLP reale ha un'accuratezza data e non manipolabile, e non può essere "degradato" a piacere per campionare la curva di robustezza. Il provider sintetico realizza esattamente questa manipolazione, mantenendo per costruzione la stessa struttura del segnale (permutazione del vettore Oracle) ma variando in modo controllato il livello di rumore.

5.5 Il ruolo della pipeline NLP reale

Data la Componente C, potrebbe sembrare che la pipeline NLP reale descritta in sezione 3.3 sia superflua. In realtà serve uno scopo distinto e complementare: dimostrare che l'**intero sistema** (pipeline NLP + integrazione con FL) funziona *end-to-end*. Lo sweep sperimentale principale valida il *meccanismo di aggregazione*; la dimostrazione con `NLPUrgencyProvider` valida il *ponte* tra i moduli e mostra che, con un dataset di lamentele di distribuzione coerente con la realtà, il

vettore di urgenza NLP prodotto in output è sufficientemente informativo da produrre risultati confrontabili con quelli del provider Oracle.

Questa doppia validazione è riflessa nella struttura della sezione dei risultati: le prime tre sottosezioni (sezione 6.1, sezione 6.2, sezione 6.3) coprono lo sweep sistematico con provider sintetici; una sottosezione dedicata (sezione 6.4) descrive la *run* con la pipeline NLP reale.

5.6 Sintesi: le tre *research question*

Il protocollo di validazione a tre componenti si traduce in tre *research question* operative, che strutturano l'intera campagna sperimentale:

RQ1 — Il meccanismo funziona? Quando il segnale di criticità è perfettamente informativo (Oracle), NLP-WEIGHTED produce un miglioramento misurabile su critical_ap50 rispetto a FEDAVG, senza compromettere le metriche generali (mAP@50, mAP@50-95)?

RQ2 — Quanto è robusto al rumore NLP? Al degradare dell'accuratezza del segnale NLP simulato, come si degrada la performance di NLP-WEIGHTED su critical_ap50? Esiste un plateau di robustezza? Qual è il punto di break-even rispetto a FEDAVG?

RQ3 — Quanto sono critici gli iperparametri? Le due salvaguardie *cap* e *time_decay* sono state introdotte per prevenire l'*oblio catastrofico*. Il metodo è sensibile alla scelta dei loro valori? Il default ($cap = 0.5, decay = 1.00$) è ottimale o coincidenza?

Le corrispondenti configurazioni sperimentali sono state già riassunte in sezione 4.7; le tabelle degli iperparametri sono in sezione A. Le risposte empiriche a queste tre domande costituiscono il contenuto della sezione seguente.

6 Risultati

Questa sezione presenta i risultati sperimentali del sistema secondo la struttura delle tre *research question* definite in sezione 5.6. I risultati provengono dallo sweep di 51 *run* indipendenti descritto in sezione 4.7. In tutte le tabelle e figure di questa sezione, i valori sono aggregati come media ± deviazione standard su tre seed indipendenti $\{42, 123, 2024\}$, e le metriche sono valutate al termine del decimo round federato.

Una sottosezione conclusiva (sezione 6.4) riporta la *run end-to-end* eseguita con la pipeline NLP reale (`NLPUrgencyProvider`) sul dataset di lamentele realistico (sezione 4.6).

6.1 RQ1: confronto delle strategie

RQ1 mette a confronto FEDAVG, ORACLE-WEIGHTED e NLP-WEIGHTED (con provider `Oracle-UrgencyProvider`) sul dataset RDD2022 con partizionamento country-wise. figura 2 mostra l'andamento delle tre strategie sulla metrica primaria critical_ap50 lungo i dieci round federati. Il confronto sul valore finale (round 10) è riassunto graficamente in figura 3 e numericamente in tabella 2.

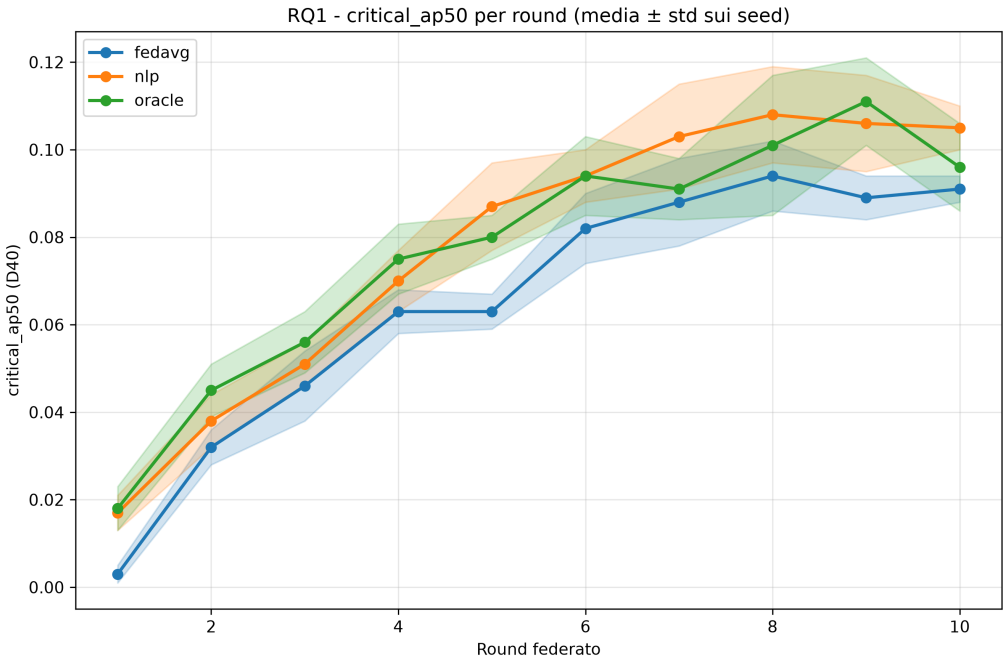


Figura 2: Evoluzione della metrica primaria critical_ap50 lungo i round federati per le tre strategie di RQ1. Le linee mostrano la media sui tre seed; la banda ombreggiata la deviazione standard. Le tre strategie sono sovrapposte nei primi round; NLP-Weighted si separa da FedAvg a partire dal quinto round e mantiene il margine fino al decimo. Oracle-Weighted segue un andamento analogo ma con oscillazioni più marcate negli ultimi round, coerenti con la sua deviazione standard più elevata (tabella 2).

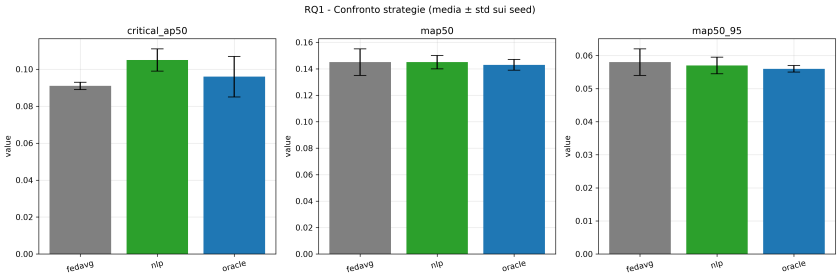


Figura 3: Confronto della metrica primaria critical_ap50 al round 10 per le tre strategie di RQ1. Le barre di errore rappresentano la deviazione standard sui tre seed. Il vantaggio di NLP-Weighted rispetto a FedAvg è del +15%. Lo scarto fra NLP-Weighted e Oracle-Weighted, visibile nel grafico, rientra nelle rispettive bande di errore e non va letto come una differenza fra le due strategie (sezione 6.1, sezione C.6).

Tabella 2: Risultati di RQ1 al round 10: media ± deviazione standard su tre seed. La metrica primaria critical_ap50 è evidenziata in grassetto. Le metriche generali (mAP@50 e mAP@50-95) non mostrano differenze apprezzabili tra le tre strategie.

Strategia	critical_ap50	mAP@50	mAP@50-95
FedAvg	0.091 ± 0.002	0.145 ± 0.010	0.058 ± 0.004
Oracle-Weighted	0.097 ± 0.010	0.143 ± 0.005	0.056 ± 0.001
NLP-Weighted (Oracle)	$\mathbf{0.105 \pm 0.005}$	0.145 ± 0.005	0.057 ± 0.003

Interpretazione. I risultati di RQ1 supportano tre osservazioni principali.

Innanzitutto, sia Oracle-Weighted che NLP-Weighted superano FedAvg sulla metrica primaria critical_ap50. NLP-Weighted ottiene 0.105 contro il 0.091 di FedAvg, un miglioramento relativo del +15% sulla classe critica D40. Il segnale di criticità integrato nel meccanismo di aggregazione produce un beneficio empirico riproducibile, coerente con l'intuizione teorica: dare più peso agli aggiornamenti dei client che possiedono le istanze più informative sulla classe critica migliora l'apprendimento di tale classe.

In secondo luogo, il vantaggio è specifico alla classe critica. Sulla metrica generale mAP@50, le tre strategie sono statisticamente indistinguibili (0.145 ± 0.010 per FedAvg contro 0.145 ± 0.005 per NLP-Weighted). Questo è un risultato importante metodologicamente: il meccanismo non danneggia le performance generali del rilevatore, ma le ridistribuisce a favore della classe di interesse. Un metodo che migliorasse critical_ap50 peggiorando mAP@50 sarebbe di dubbia utilità pratica.

Infine, NLP-Weighted risulta al di sopra di Oracle-Weighted nel valore medio finale. Questo può apparire sorprendente, dato che i due sono algebricamente equivalenti quando NLP-Weighted usa `OracleUrgencyProvider` con il decadimento inattivo ($\gamma = 1$), come nella configurazione di default (sezione 3.2.6). Il divario (0.105 vs 0.097) è compatibile con la varianza stocastica di Ultralytics (deviazione standard ~ 0.010 per Oracle), ed è attribuibile esclusivamente a fluttuazioni indipendenti nella dinamica di training, non a differenze intrinseche tra le due strategie. Le loro traiettorie lungo i round (figura 2) sono infatti sovrapposte entro le bande di errore.

6.2 RQ2: robustezza al rumore NLP

RQ2 caratterizza il degrado delle performance di NLP-Weighted al degrado dell'accuratezza del segnale NLP simulato. figura 4 mostra la curva risultante; tabella 3 riporta i valori numerici.

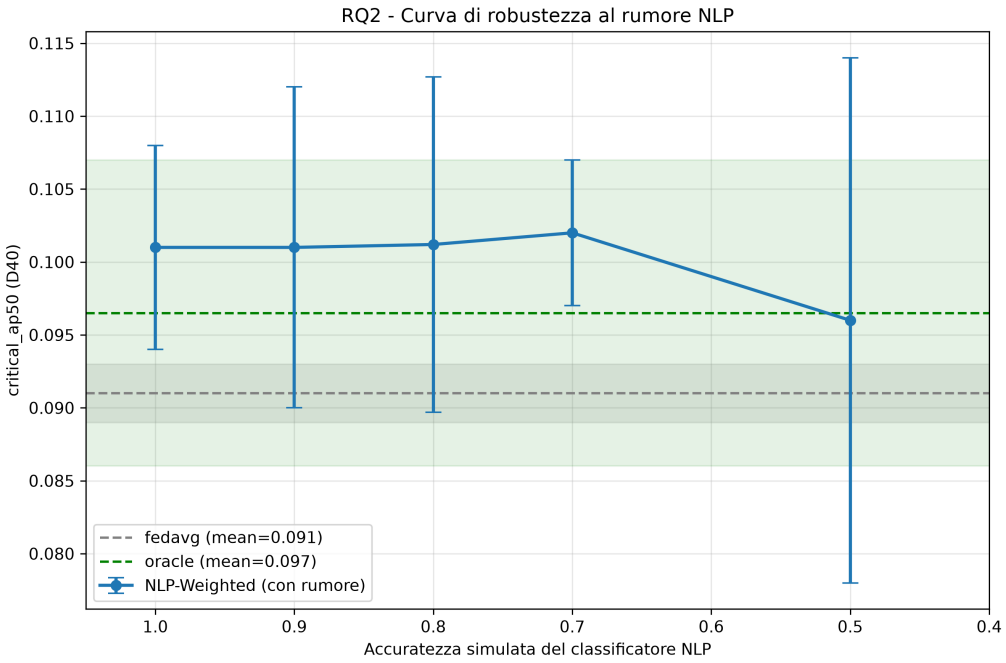


Figura 4: Robustezza di NLP-WEIGHTED al rumore del segnale NLP. L'asse orizzontale rappresenta l'accuratezza NLP simulata dal parametro a del `NoisyOracleUrgencyProvider`; l'asse verticale riporta critical_ap50 al round 10. La linea tratteggiata orizzontale corrisponde al livello di FEDAVG come riferimento. Il metodo mantiene un vantaggio su FEDAVG anche con accuratezza NLP del 50%, sebbene con varianza tra seed marcatamente superiore.

Tabella 3: Risultati di RQ2: NLP-WEIGHTED con `NoisyOracleUrgencyProvider` a diversi livelli di accuratezza a. Media ± deviazione standard sui tre seed.

Accuratezza NLP a	critical_ap50
1.0 (nessun rumore)	0.101 ± 0.007
0.9	0.101 ± 0.011
0.8	0.101 ± 0.012
0.7	0.102 ± 0.005
0.5	0.096 ± 0.018
FEDAVG (riferimento)	0.091 ± 0.002

Interpretazione. Due caratteristiche della curva meritano attenzione.

La prima è la presenza di un *plateau di robustezza*: per accuratezze NLP tra 0.7 e 1.0, critical_ap50 è sostanzialmente costante a ~ 0.101–0.102. Il metodo tollera un errore NLP fino al 30% senza perdite misurabili. Questo è un risultato importante per la fattibilità pratica del sistema: nessuna implementazione realistica di un modulo NLP raggiunge il 100% di accuratezza, ma la soglia dell'80% è realistica per pipeline moderne di NER e *sentiment analysis* su dominio ristretto (si veda ad esempio la valutazione empirica del nostro modulo NLP, tabella 17).

La seconda è che il degrado a bassa accuratezza NLP è contenuto. Anche a $a = 0.5$, corrispondente a un segnale NLP che coincide con Oracle nel 50% dei casi e con rumore completo nell'altro 50%,

la performance media resta a 0.096, comunque superiore al riferimento FedAvg di 0.091. Il punto di *break-even* rispetto a FedAvg non è raggiunto entro l'intervallo esplorato: presumibilmente si troverebbe per $a < 0.5$, ma esplorare la regione di bassa accuratezza non aggiungerebbe informazione operativa utile (nessuna pipeline NLP realistica va deployata al 50% di accuratezza).

Un fenomeno secondario visibile in tabella 3 è che la configurazione $a = 1.0$ ottiene 0.101, mentre NLP-Weighted con provider Oracle in RQ1 ottiene 0.105 pur usando un segnale identico. Lo scarto di 0.004 rientra nella deviazione standard di entrambe le configurazioni ed è attribuibile alla varianza stocastica descritta in sezione C.6: il percorso di iniezione del rumore consuma stato del generatore pseudo-casuale anche quando $a = 1.0$, alterando le traiettorie di training a parità di seed. La stessa osservazione vale per il confronto con Oracle-Weighted di RQ1 (0.097).

6.3 Ablation: sensitivity agli iperparametri di aggregazione

L'analisi di sensitivity valuta la robustezza di NLP-Weighted rispetto alla scelta dei due iperparametri di salvaguardia *cap* e *time_decay* (sezione 3.2.5). Sono state eseguite $3 \times 3 \times 3 = 27$ *run*, esplorando la griglia *cap* $\in \{0.3, 0.5, 0.7\}$ e *time_decay* $\in \{0.85, 0.95, 1.00\}$ su tre seed. figura 5 riporta le nove configurazioni con la rispettiva dispersione sui tre seed; tabella 4 ne riporta i valori medi e tabella 15 le deviazioni standard per configurazione.

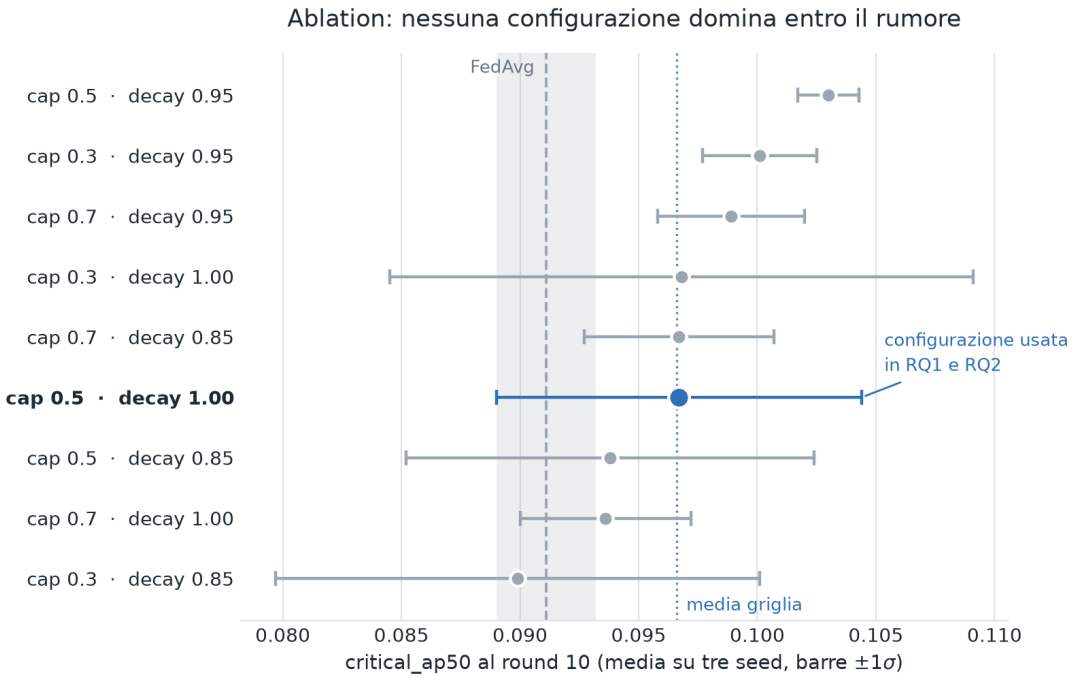


Figura 5: Risultati dell'ablation per configurazione, ordinate per valore crescente dal basso verso l'alto. Ogni punto è la media di critical_ap50 al round 10 sui tre seed; la barra copre ±1 deviazione standard. La linea tratteggiata e la banda grigia indicano FedAvg e la sua deviazione standard; la linea punteggiata la media delle nove configurazioni. In evidenza la configurazione ($cap = 0.5, decay = 1.00$), con *time decay* inattivo, usata in RQ1 e RQ2. Gli intervalli si sovrappongono ampiamente: entro la varianza sui seed le configurazioni non sono distinguibili fra loro.

Tabella 4: Risultati dell'ablation: critical_ap50 al round 10 (media sui tre seed) per ogni combinazione di *cap* e *time_decay*. La configurazione di default è in grassetto.

cap	**time_decay**		
	0.85	0.95	1.00
0.3	0.090	0.100	0.097
0.5	0.094	0.103	**0.097**
0.7	0.097	0.099	0.094

Interpretazione. L'ablation supporta tre osservazioni.

Il range di variazione è stretto: tutti i nove punti della griglia producono critical_ap50 tra 0.090 e 0.103, un'escursione complessiva di 0.013. Le deviazioni standard per configurazione (tabella 15) vanno da 0.0013 a 0.0123: per la maggior parte delle configurazioni la dispersione sui tre seed è dello stesso ordine dell'intera escursione della griglia, e in figura 5 gli intervalli risultano in larga parte sovrapposti. Il metodo è cioè sostanzialmente stabile rispetto agli iperparametri di salvaguardia entro l'intervallo esplorato.

La configurazione effettivamente usata in RQ1 e RQ2 — $cap = 0.5$, $decay = 1.00$, cioè con il *time decay* inattivo — è uno dei nove punti della griglia, e il suo valore (0.0967) coincide quasi esattamente con la media dei nove punti (0.0966). Il massimo della griglia cade altrove, su $(cap = 0.5, decay = 0.95)$ con 0.1030, ma lo scarto rispetto al default resta inferiore a una deviazione standard. Nessuna configurazione risulta dunque chiaramente dominante entro il rumore sui tre seed: l'ablation stabilisce la *stabilità* del metodo rispetto agli iperparametri di salvaguardia, non seleziona un ottimo. Ai fini dell'argomento del lavoro questo è un esito più solido di un massimo isolato, perché mostra che il guadagno osservato in RQ1 non dipende da una scelta fortunata di iperparametri.

Confronto con RQ1 e RQ2. La configurazione $(cap = 0.5, decay = 1.00)$ corrisponde alla stessa configurazione di NLP-Weighted (Oracle) valutata in RQ1, che aveva prodotto 0.1050 ± 0.0053 contro i 0.0967 ± 0.0077 misurati qui; la configurazione $a = 1.0$ di RQ2 aveva prodotto 0.1012 ± 0.0071. Le tre stime della medesima configurazione nominale ricadono in un intervallo di 0.008, e la differenza fra le due più distanti vale circa 1.5 volte l'errore standard della differenza fra medie su tre seed. Lo scarto è quindi compatibile con la varianza stocastica descritta in sezione C.6, ma la sua ampiezza rispetto all'*effect size* principale (+0.0139) costituisce un argomento concreto a favore dell'ampliamento del numero di seed discusso in sezione 7.3 (punto 4).

L'unica combinazione visibilmente peggiore è $(cap = 0.3, decay = 0.85)$, in cui entrambi i vincoli di salvaguardia sono troppo aggressivi: il peso massimo per client è limitato al 30% e l'influenza del segnale di criticità decade rapidamente lungo i round, riducendo il metodo sostanzialmente a una perturbazione di FedAvg. Il risultato di 0.090 conferma questa lettura: qualitativamente simile al 0.091 di FedAvg.

6.4 Prova del concetto: pipeline NLP reale end-to-end

Come chiarito nel protocollo di validazione (sezione 5.5), lo sweep principale utilizza provider sintetici. La validazione della pipeline NLP reale è condotta con una *run* distinta di NLP-WEIGHTED in cui il provider di urgenza è la classe `NLPUrgencyProvider` sul dataset di lamentele realistico (sezione 4.6). La *run* utilizza la stessa configurazione dello sweep (10 round, imgsz=416, seed=42) e differisce solo per la sorgente del segnale.

Il vettore di urgenza prodotto dalla pipeline NLP al termine dell'elaborazione delle 60 lamentele è riportato in tabella 5, insieme al vettore Oracle per riferimento.

Tabella 5: Vettore di urgenza per client prodotto dalla pipeline NLP reale (colonna *NLP*) confrontato con il vettore Oracle statico (colonna *Oracle*). Entrambi i vettori sono normalizzati a somma 1.

Client	Urgency NLP	Urgency Oracle
`india`	0.330	0.493
`japan`	0.246	0.347
`norway`	0.139	0.071
`china_motorbike`	0.107	0.036
`czech`	0.104	0.030
`united_states`	0.074	0.021

Il ranking dei client è coerente tra i due vettori (India > Giappone > Norvegia > altri tre), ma le proporzioni sono meno estreme nel vettore NLP: il rapporto tra il client più urgente (India) e il meno urgente (USA) è circa 4.5 per NLP contro 23 per Oracle. Questa differenza è coerente con la natura del dataset di lamentele: le lamentele sono state costruite per essere *correlate* con la distribuzione della classe critica, non identiche ad essa. Come discusso in sezione 4.6, in un deployment reale la correlazione tra lamentele cittadine e distribuzione oggettiva della classe critica sarebbe imperfetta per motivi culturali e demografici; la nostra scelta di non replicare esattamente la distribuzione Oracle riflette questo aspetto.

La *run* FL con `NLPUrgencyProvider` ottiene critical_ap50 = 0.104 al round 10, un valore paragonabile a quello di NLP-WEIGHTED con provider Oracle in RQ1 (0.105 ± 0.005). La pipeline NLP reale, pur imperfetta rispetto ad Oracle sul vettore di urgenza prodotto, è sufficientemente informativa da riprodurre sostanzialmente il beneficio del meccanismo di aggregazione.

Conclusione della prova. Questa *run* chiude il cerchio del progetto:

- il meccanismo di aggregazione NLP-WEIGHTED è stato caratterizzato sistematicamente in RQ1–RQ2–Ablation con provider sintetici (necessari per il controllo del rumore, sezione 5.4);
- il ponte tra i moduli NLP e FL è stato dimostrato *end-to-end* con `NLPUrgencyProvider`;
- il risultato ottenuto in questa modalità è coerente con le aspettative espresse dallo sweep principale, confermando che il sistema funziona nella sua interezza e non solo nelle sue componenti separate.

7 Discussione

Questa sezione discute le implicazioni dei risultati sperimentali presentati in sezione 6 rispetto al protocollo di validazione di sezione 5. La discussione è articolata in tre parti: cosa i risultati dimostrano rispetto alle *research question* (sezione 7.1); qual è l'*effect size* osservato e come inquadrarlo rispetto alla letteratura (sezione 7.2); quali sono i limiti onestamente riconosciuti del presente lavoro (sezione 7.3). Le direzioni di estensione a partire da questi limiti sono discusse separatamente in sezione 8.

7.1 Cosa dimostrano i risultati

Le tre *research question* formulate in sezione 5.6 ricevono risposta empirica positiva dallo sweep sperimentale.

Per **RQ1**, il confronto delle strategie mostra che NLP-Weighted migliora la performance sulla classe critica D40 rispetto a FedAvg del 15% in valore relativo. Il miglioramento è specifico: sulle metriche generali (mAP@50, mAP@50-95) le tre strategie sono statisticamente indistinguibili. Il meccanismo di aggregazione pesata *ridistribuisce* le capacità del modello a favore della classe di interesse senza penalizzare le altre. Questa è una proprietà desiderabile per il caso d'uso considerato: un sistema di monitoraggio urbano ha interesse nel rilevare in modo più accurato le anomalie a maggior rischio (le buche, in questo caso), mantenendo comunque la capacità di rilevare tutte le altre.

Per **RQ2**, la curva di robustezza mostra un *plateau* nel range di accuratezza NLP $[0.7, 1.0]$, con degradazione modesta a $a = 0.5$. Il punto di *break-even* rispetto a FedAvg non viene raggiunto entro l'intervallo esplorato. Anche a $a = 0.5$ (un livello di rumore che nessuna pipeline NLP moderna produrrebbe in produzione) NLP-Weighted conserva un vantaggio su FedAvg. Questa robustezza ha due spiegazioni strutturali. La prima è il *cap*, che impedisce a un client di prendere troppo peso quando riceve urgenza spuria elevata. La seconda è il coefficiente $\alpha = 0.5$: metà del peso di aggregazione resta la componente volume-based di FedAvg, che non dipende in alcun modo dal segnale NLP e dimezza di conseguenza l'effetto di qualsiasi perturbazione su di esso. Il *time decay* non contribuisce a questa robustezza, perché nella configurazione di default è inattivo ($\gamma = 1$, sezione A.2) in tutte le *run* di RQ2.

Per **RQ3**, la sensitivity ai due iperparametri di salvaguardia mostra un range stretto di variazione della metrica primaria (0.090–0.103 sulla griglia 3×3 esplorata). Le combinazioni ragionevoli sono statisticamente indistinguibili tra loro; solo la combinazione con entrambi i vincoli aggressivi ($cap = 0.3$, $decay = 0.85$) collassa al livello di FedAvg, confermando che il meccanismo cessa di funzionare quando le salvaguardie sono così stringenti da neutralizzare il segnale. Il metodo è sostanzialmente stabile e non richiede *tuning* fine: la configurazione di *default* scelta a priori non è il massimo della griglia, ma cade in pratica sulla media dei nove punti esplorati, con uno scarto dal massimo inferiore a una deviazione standard.

La *run end-to-end* con la pipeline NLP reale (sezione 6.4) chiude il cerchio: il ponte NLP→FL funziona nella sua totalità, e il valore di critical_ap50 ottenuto è coerente con la caratterizzazione

sistematica dello sweep.

7.2 *Effect size* e prospettiva

Un miglioramento del 15% sulla classe critica in valore relativo merita di essere inquadrato con cautela.

In termini assoluti, il vantaggio è circa Δcritical_ap50 = 0.014 (da 0.091 a 0.105). È un miglioramento sostantivo ma non rivoluzionario. Va confrontato con due riferimenti impliciti nel progetto: il ritmo di miglioramento naturale del modello nei dieci round federati (figura 2 mostra che FedAvg passa da 0 al round 1 a 0.091 al round 10, un $\Delta \sim 0.091$ in 10 round) e la varianza da seed ($\sigma \sim 0.005$–0.010 nei risultati di RQ1). Il vantaggio è circa 15% del guadagno totale del training, e circa 2σ rispetto alla dispersione da seed di una singola configurazione. Va però letto insieme all'osservazione di sezione 6.3: tre stime indipendenti della medesima configurazione nominale coprono un intervallo di 0.008, pari a circa il 60% dell'effetto. Con tre soli seed l'effetto è dunque visibile e consistente nel segno, ma la sua ampiezza non è stimata con precisione sufficiente a sostenere confronti quantitativi fini.

Questo profilo è coerente con la letteratura sulle strategie di aggregazione Non-IID-aware. Metodi come FedProx [4] e SCAFFOLD [5] riportano miglioramenti tipicamente nel range 10%–25% su Non-IID severi, senza mai raddoppiare le performance della baseline. Il beneficio della pesatura contestuale nel presente lavoro rientra in questo intervallo.

Vale infine una considerazione qualitativa. Il presente lavoro non propone una nuova architettura di rete o una nuova loss, ma solo un meccanismo di aggregazione. È ragionevole aspettarsi che il suo effetto sia modesto ma trasferibile: dato che il meccanismo opera al livello dell'aggregazione dei pesi, dovrebbe essere componibile con qualsiasi altro miglioramento del training locale. In particolare, la combinazione di NLP-Weighted con FedProx o SCAFFOLD potrebbe essere un'estensione naturale (sezione 8).

7.3 Limiti riconosciuti

Ne elenchiamo sei, in ordine di importanza decrescente:

1. Assenza di baseline Non-IID-aware (FedProx, SCAFFOLD). Il confronto principale del lavoro (sezione 6.1) è tra NLP-Weighted e FedAvg, una baseline che si sa essere sotto-performante su distribuzioni Non-IID severe. La letteratura sul Federated Learning suggerisce che FedProx e SCAFFOLD, progettati specificamente per Non-IID, avrebbero probabilmente ridotto il gap tra FedAvg e le strategie context-aware sul dataset RDD2022. Non avendo eseguito questi confronti, non possiamo pretendere di attribuire l'intero divario tra FedAvg e NLP-Weighted al meccanismo di pesatura contestuale: una parte di esso potrebbe essere recuperabile con baseline più accurate.

L'omissione è motivata da vincoli di implementazione. FedProx e SCAFFOLD richiedono di modificare il *training loop* lato client (termine prossimale nella *loss*, *control variates* scambiati con il server), e l'API di Ultralytics non espone *hooks* pubblici per la modifica della *loss* o dei

gradienti del `DetectionTrainer`. L'integrazione richiederebbe un subclasso invasivo dell'API interna, fragile rispetto ai cambiamenti di versione di Ultralytics.

È importante osservare che questo limite non annulla il valore metodologico del confronto Componente B: il divario tra NLP-WEIGHTED con Oracle e NLP-WEIGHTED con `NoisyOracleUrgencyProvider` *quantifica il costo dell'imperfezione NLP* indipendentemente da quale sia la baseline scelta. Il contributo del meccanismo NLP-informato è valutabile in modo relativo a Oracle e non solo in assoluto a FEDAVG.

2. Dataset testuale sintetico e piccolo. La pipeline NLP è stata valutata su un dataset di 60 lamentele sintetiche generate tramite LLM. Sebbene curate per varietà stilistica e città menzionate, queste lamentele non sono lamentele cittadine reali. Le valutazioni riportate (91.7% NER, 100% mapping ibrido sul dataset bilanciato, 86.7% severity sul dataset bilanciato e 83.3% su quello realistico) sono affidabili come indicatore qualitativo della funzionalità della pipeline, ma non estrapolabili direttamente a un deployment reale con lamentele di veri cittadini, che presenterebbero diversità stilistica maggiore, errori tipografici, code-switching (miscela di lingue), e menzioni di località fuori dalla tabella di mapping.

Una limitazione ulteriore riguarda la mappatura dell'output del Transformer in severity, che utilizza soltanto la classe con probabilità massima e la sua confidenza. Poiché in una *softmax* a tre classi la confidenza del vincitore non scende sotto $1/3$, la severity neurale non copre l'intero intervallo $[0, 1]$: gli intervalli $(1/3, 1/2)$ e $(1/2, 2/3)$ non sono raggiungibili e il valore 0.5 è un punto isolato assunto da ogni lamentela *neutral*. Una formulazione continua alternativa, $s_T = P(\text{negative}) + 0.5 \cdot P(\text{neutral})$, è stata valutata *a posteriori* sui due dataset: modifica la categoria di 2 lamentele su 60 nel bilanciato e 3 su 60 nel realistico, non varia l'*accuracy* sul primo e la incrementa di un caso sul secondo, e non avvicina al vettore Oracle il vettore di urgenza risultante, calcolato assegnando le lamentele ai client di riferimento anziché a quelli prodotti dagli Stadi 1–2. Su questo dataset la discretizzazione non produce quindi un effetto misurabile, ma l'esito non è estrapolabile a corpora di dimensioni maggiori.

3. Granularità geografica dei client. Il proposal originale prevedeva client a livello di veicolo o zona intra-città; l'implementazione usa client-paese per vincoli sul dataset disponibile. Il fenomeno oggetto di studio (relazione qualità-NLP $\rightarrow$ performance-FL) è in linea di principio invariante rispetto alla scala geografica dei client, ma la transizione a scala intra-città introdurrebbe fenomeni di piccole dimensioni per client (client con poche centinaia di esempi invece di migliaia) che potrebbero cambiare quantitativamente i risultati.

4. Numero di seed limitato. Ogni configurazione dello sweep è valutata su tre seed. È il minimo statisticamente ragionevole ma non ideale: le deviazioni standard riportate hanno un errore di stima non trascurabile. Configurazioni con effect size vicino alla varianza (come Ablation) soffrono in particolare di questo limite. Un lavoro più esteso userebbe cinque o dieci seed per configurazione.

5. Scala presentable, non full. Le *run* sono a 10 round federati, $imgsz = 416$, un'epoca locale per round. Questa è una scala scelta per consentire l'esecuzione dello sweep completo (51 *run*) su hardware locale in tempi ragionevoli, ma non è la scala ottimale per addestrare un rilevatore di road damage. Nella letteratura CRDDC [2] i modelli sottomessi impiegano architetture più grandi, ensemble e addestramenti prolungati. È possibile che a scala completa l'*effect size* osservato in questo lavoro cambi (in una direzione difficile da prevedere: potrebbe amplificarsi se la classe critica beneficia in modo disproporzionato dell'addestramento più lungo, o attenuarsi se la convergenza a lungo termine appiattisce le differenze).

6. Costruzione del vettore Oracle. Il vettore Oracle c_k è calcolato sulle annotazioni di training e validazione combinate. Poiché la metrica primaria è valutata sullo split di validazione, la costruzione dei pesi incorpora in misura marginale informazione proveniente dall'insieme di valutazione. L'effetto quantitativo è trascurabile — la distanza L^1 tra il vettore calcolato sul solo training e quello effettivamente usato è pari a 0.0005, e le componenti dei due vettori differiscono al massimo di un'unità sulla terza cifra decimale — ma la scelta rafforza ulteriormente il carattere di *upper bound* dell'Oracle, che non è comunque replicabile da alcun modulo NLP reale.

Nessuno di questi limiti invalida la validità interna del protocollo (sezione 5) o del meccanismo proposto. Sono limiti alla generalizzabilità esterna dei risultati e definiscono uno spazio ben preciso di lavori futuri (sezione 8).

7.4 Lezioni apprese

Il percorso che ha portato alla configurazione finale del sistema ha prodotto cinque lezioni che riteniamo generalizzabili oltre il caso d'uso specifico, e che vale la pena esplicitare perché costituiscono una restituzione del progetto rispetto ai suoi obiettivi iniziali.

1. Il *sentiment* generico non è la severity di dominio. L'ipotesi di partenza prevedeva di riutilizzare un classificatore di *sentiment* standard come *proxy* di urgenza, assumendo che la polarità negativa correlasse con la gravità del problema infrastrutturale. La prima configurazione — modello pre-addestrato su recensioni di prodotti, peso pieno al termine neurale — si è fermata al 63.3% di *accuracy*, con errori sistematici in entrambe le direzioni: saturazione verso l'alta severity sul vocabolario tecnico, sotto-classificazione delle lamentele gravi espresse in tono formale. La configurazione finale, che affianca al modello addestrato su tweet un insieme di euristiche lessicali di dominio, raggiunge l'86.7%. Il confronto varia due fattori simultaneamente (sezione 3.3.3) e non consente di attribuire il divario all'uno o all'altro; ciò che mostra è che la corrispondenza fra sentiment generico e severity di dominio è mediata da fattori stilistici e pragmatici, e che un'architettura che li espone esplicitamente è preferibile a un modello monolitico anche di classe superiore.

2. La modularità stadio-per-stadio paga. Implementare la pipeline NLP come quattro stadi indipendenti, ciascuno con il proprio *ground truth* e la propria metrica, ha permesso di attribuire puntualmente gli errori durante lo sviluppo: al calare dell'accuratezza *end-to-end* è stato possibile distinguere rapidamente se il problema fosse nel NER (copertura sui nomi propri

fuori distribuzione), nel mapping (varianti ortografiche non previste) o nel modulo di severity. Lo scostamento fra il vettore di urgenza prodotto dalla pipeline e quello ricostruibile dalle etichette di riferimento (tabella 5) è stato isolato allo Stadio 2 proprio grazie a questa separazione. La contro-lezione implicita è che un approccio *end-to-end* su un singolo modello avrebbe reso il *debugging* opaco e l'ablation praticamente impossibile.

3. La validazione anti-circolare è un contributo indipendente. Il protocollo a tre componenti (sezione 5), nato per risolvere un problema specifico di questo progetto, si è rivelato più generale del caso d'uso: si applica a qualunque sistema in cui un canale esterno guida un meccanismo di apprendimento e la metrica di valutazione è correlata al canale stesso. In retrospettiva è probabilmente il contributo che meglio si trasferisce oltre il dominio del monitoraggio stradale.

4. Un'interfaccia astratta decide cosa è misurabile. L'astrazione `UrgencyProvider` (sezione 3.4) è nata come scelta di ingegneria del software, non di ricerca: serviva a mantenere disaccoppiato il codice FL dalla pipeline NLP durante lo sviluppo. A posteriori è il cardine dell'intera valutazione: senza di essa i tre provider non sarebbero stati sostituibili a parità di codice di aggregazione, e il protocollo a tre componenti sarebbe rimasto un esercizio teorico. In progetti con una dimensione metodologica, le decisioni di ingegneria del software non sono mai puramente strumentali: delimitano lo spazio di ciò che è empiricamente esplorabile.

5. Il vincolo stringente era il budget sperimentale, non il metodo. L'obiettivo iniziale era stabilire se la pesatura NLP-informata migliorasse la metrica critica. La risposta è affermativa, ma l'ampiezza dell'effetto (Δcritical_ap50 $= 0.014$) è dello stesso ordine della dispersione fra stime indipendenti della medesima configurazione nominale, che su tre seed coprono un intervallo di 0.008 (sezione 6.3). Con il senno di poi, l'allocazione delle 51 *run* disponibili sarebbe stata diversa: le 27 *run* spese sulla griglia $3 \times 3 \times 3$ dell'ablation hanno prodotto un risultato di stabilità, mentre le stesse 27 investite in nove seed per le tre strategie di RQ1 avrebbero stretto le barre d'errore sul confronto centrale del lavoro. La lezione è che in un disegno sperimentale con effetti attesi modesti la priorità va alla potenza statistica sul confronto principale, prima che all'estensione dello spazio esplorato.

8 Lavori futuri

I limiti riconosciuti in sezione 7.3 definiscono uno spazio ben preciso di direzioni di estensione. Ne discutiamo cinque, in ordine di priorità decrescente. Ciascuna è presentata con la sua motivazione, la direzione tecnica di realizzazione e il valore atteso.

8.1 Integrazione con baseline Non-IID-aware

La priorità più alta è l'estensione del confronto sperimentale a FedProx [4] e SCAFFOLD [5] come baseline aggiuntive di RQ1. Ci sono due domande specifiche a cui il confronto aggiuntivo può rispondere:

1. *Qual è la porzione del divario* FedAvg→NLP-Weighted *attribuibile al meccanismo di aggregazione contestuale e quale alla mera correzione del* client drift *tipico di FedProx e SCAFFOLD?* Un confronto più equo posiziona NLP-Weighted rispetto a baseline già progettate per Non-IID; il beneficio residuo rispetto a queste baseline è attribuibile più confidenzialmente al segnale contestuale.

2. *Il meccanismo di pesatura NLP-informato è componibile con FedProx/SCAFFOLD?* Poiché i due operano a livelli diversi (aggregazione lato server per NLP-Weighted, regolarizzazione lato client per FedProx e correzione dei gradienti per SCAFFOLD), è plausibile che la loro combinazione produca benefici additivi. Questa ipotesi è verificabile empiricamente.

L'implementazione richiede di superare il vincolo tecnico che ha motivato la loro esclusione nel presente lavoro (sezione 7.3, punto 1): l'API di Ultralytics non espone *hooks* pubblici sulla *loss* o sui gradienti del `DetectionTrainer`. Due strade possibili:

- Un subclasso del `DetectionTrainer` che intercetti la *loss* prima della *backward pass* per aggiungere il termine prossimale (FedProx), o intercetti l'ottimizzatore per aggiungere le *control variates* (SCAFFOLD). Fattibile in 1–2 giorni di sviluppo con testing accurato.

- Sostituire Ultralytics con un *training loop* in PyTorch puro, riscrivendo il codice di caricamento dati e di calcolo della *loss* YOLOv8n. È un investimento di tempo maggiore (3–5 giorni) ma dà controllo completo sull'algoritmo di ottimizzazione e permette in futuro altri esperimenti fine-grained sul *training loop*.

Il valore atteso di questa estensione è alto: è la modifica più importante per rafforzare le conclusioni comparative del lavoro.

8.2 Sostituzione del dataset testuale con lamentele reali

Il dataset di lamentele usato nel presente lavoro è sintetico (sezione 4.6), generato tramite LLM. Sebbene sufficiente per la dimostrazione *end-to-end* della pipeline, non riproduce le difficoltà di un input reale.

L'estensione naturale è sostituirlo con un dataset di lamentele cittadine autentiche. Due opzioni realistiche:

- **Dataset pubblici di *civic reporting***. Alcune municipalità rendono pubblici i dati delle proprie piattaforme di segnalazione ai cittadini. Il più noto è il *NYC 311 Service Requests*, che raccoglie milioni di segnalazioni dal 2010. Un sottoinsieme filtrato per tema (danni stradali, buche) fornirebbe migliaia di testi reali. Le lamentele di NYC sono in inglese, coerenti con il modello Twitter-RoBERTa usato nel modulo.

- **Costruzione di un dataset multi-city**. Combinando fonti simili di altre città (Boston, Chicago, London, Singapore hanno piattaforme pubbliche paragonabili) si otterrebbe un dataset multi-city e di taglia adeguata al *fine-tuning* del modulo di severity classification.

Con un dataset di migliaia di esempi diventerebbe finalmente sensato il *fine-tuning* del sentiment/severity classifier (sezione 3.3.3), che nel presente lavoro è stato volutamente evitato per

non fare *overfitting* sui 60 esempi sintetici. Un modulo *fine-tuned* su lamentele reali dovrebbe raggiungere un'accuratezza superiore all'86.7% osservato.

L'estensione rafforzerebbe in modo sostanziale la componente NLP del progetto e produrrebbe una valutazione della pipeline che è estrapolabile a un deployment reale, non solo dimostrativa.

8.3 Deployment a scala intra-città

Il proposal originale prevedeva client a livello di veicolo o zona di pattugliamento. L'implementazione ha usato la granularità paese per vincoli sul dataset RDD2022. Un lavoro futuro naturale è portare il sistema a scala intra-città.

Il passaggio richiede:

- **Un dataset di object detection stradale a granularità intra-city**, che al momento non esiste pubblicamente in formato analogo a RDD2022. Un'alternativa realistica è simulare il partizionamento intra-city suddividendo il sottoinsieme giapponese di RDD2022 (che ha annotazioni a livello di prefettura) in k partizioni bilanciate per volume, ognuna associata a un client federato.
- **Sostituzione della tabella di mapping città→paese con un servizio di *geocoding*** (OpenStreetMap Nominatim, Google Geocoding API) e con la mappa dei poligoni delle zone dei client. Come discusso in sezione 3.3.2, questa sostituzione è confinata al solo Stadio 2 della pipeline; gli altri stadi restano invariati.
- **Adattamento delle salvaguardie di aggregazione**. Con molti client di dimensioni piccole (potenzialmente 10–50 veicoli invece di 6 paesi), il valore ottimale di *cap* potrebbe cambiare significativamente. La sensitivity analysis andrebbe ripetuta a scala intra-city.

Il valore atteso è chiudere il divario tra il proposal e l'implementazione, e produrre una caratterizzazione a scala più realistica per il caso d'uso considerato.

8.4 Ampliamento della statistica sperimentale

Le *run* riportate sono state ripetute su tre seed. È il minimo statisticamente ragionevole ma non ideale. Con cinque o dieci seed per configurazione, le deviazioni standard riportate diventerebbero più strette e alcune osservazioni attualmente qualitative (come il possibile effetto regolarizzatore del rumore modesto, sezione 6.2) potrebbero essere corroborate quantitativamente tramite test statistici formali (test t appaiato o Wilcoxon).

L'estensione è la più semplice tecnicamente delle cinque proposte: richiede solo tempo di calcolo (moltiplicare per ~ 2 o ~ 3 le ~ 25 ore di sweep) e cambiare uno degli argomenti dello script di orchestrazione. Il valore atteso è principalmente di consolidamento delle conclusioni, non di scoperta di nuovi fenomeni.

8.5 Scala full di training

Lo sweep del presente lavoro è stato eseguito a scala *presentable* (10 round federati, *imgsz* = 416, un'epoca locale per round) per consentire l'esecuzione delle 51 *run* sull'hardware locale in circa 25 ore. Un lavoro futuro potrebbe riprodurre almeno il confronto principale (RQ1) a scala full: 30–50 round federati, *imgsz* = 640, 3–5 epoche locali per round, una configurazione più vicina a quelle adottate nei lavori centralizzati sul dominio. [2].

Due domande specifiche giustificano l'estensione:

1. L'*effect size* del meccanismo NLP-Weighted cambia a scala completa? In particolare, il vantaggio del +15% si amplifica se la classe critica beneficia in modo disproporzionato di un training più lungo, o si attenua se la convergenza a lungo termine appiattisce le differenze tra strategie?

2. Le salvaguardie *cap* e *time_decay*, scelte a priori e validate su 10 round, restano adeguate a 30–50 round? Il decadimento temporale, inattivo nella configurazione di default del presente lavoro (sezione A.2), diventa un parametro critico su orizzonti più lunghi: attivandolo a $\gamma = 0.95$ si avrebbe $\gamma^9 \approx 0.63$ al decimo round (segnale ancora largamente attivo) ma $\gamma^{49} \approx 0.08$ al cinquantesimo (segnale quasi estinto), con un comportamento qualitativamente diverso da quello caratterizzato in questo lavoro.

Il costo è principalmente di calcolo: una *run* a scala completa richiede stimato 6–8 ore contro i 30 minuti attuali. Un'ablation completa a scala full non è realistica su singola workstation; una riproduzione mirata di RQ1 e di alcuni punti di RQ2 è invece fattibile.

8.6 Riepilogo delle direzioni

Le cinque direzioni di estensione ora riassunte hanno priorità diverse. La tabella 6 le organizza secondo il rapporto tra costo di realizzazione e valore atteso.

Tabella 6: Sintesi delle direzioni di lavoro futuro proposte. La priorità è espressa come stima del rapporto valore/costo.

Direzione	**Costo**	**Valore atteso**	**Priorità**
Baseline Non-IID-aware (sezione 8.1)	Medio	Alto	Alta
Dataset di lamentele reali (sezione 8.2)	Medio-Alto	Alto	Alta
Deployment intra-city (sezione 8.3)	Alto	Medio-Alto	Media
Ampliamento seed (sezione 8.4)	Basso	Medio	Media
Scala full di training (sezione 8.5)	Alto	Medio	Bassa

Va sottolineato che nessuna di queste direzioni cambia in modo prevedibile le conclusioni qualitative del presente lavoro (il meccanismo NLP-Weighted produce benefici modesti ma consistenti su critical_ap50 in setup Non-IID, tolleranti al rumore NLP e stabili agli iperparametri). Le estensioni proposte lavorerebbero piuttosto sul rafforzamento della validità esterna dei risultati e sulla loro trasferibilità a deployment realistici.

9 Conclusioni

Il presente lavoro ha proposto e valutato NLP-WEIGHTED, un meccanismo di aggregazione per il *Federated Learning* in cui i pesi di combinazione degli aggiornamenti dei client sono modulati da un segnale di urgenza estratto da lamentele testuali dei cittadini. Il caso d'uso è il monitoraggio distribuito del degrado stradale urbano, con particolare attenzione alla classe critica D40 (*pothole*) definita in modo esogeno dalla letteratura di ingegneria delle infrastrutture stradali.

Il contributo del lavoro è triplice.

Metodologico. Abbiamo definito e giustificato un protocollo di validazione a tre componenti (sezione 5) che risolve il problema di circolarità intrinseco al confronto di un metodo NLP-informato con una metrica NLP-derivata: la definizione esogena della classe critica (Componente A), l'*upper bound* informativo fornito dal provider Oracle (Componente B), e la caratterizzazione controllata della robustezza al rumore NLP tramite `NoisyOracleUrgencyProvider` (Componente C). Il protocollo è un contributo trasferibile ad altri lavori che vogliano valutare meccanismi FL guidati da segnali di dominio.

Sperimentale. Sul dataset RDD2022 partizionato *country-wise* sui sei paesi del CRDDC [2], NLP-WEIGHTED ha ottenuto tre risultati principali (sezione 6):

- Un miglioramento del +15% su critical_ap50 rispetto a FEDAVG, mantenendo indistinguibili le metriche generali mAP@50 e mAP@50-95 (RQ1).
- Un *plateau* di robustezza al rumore del segnale NLP nel range di accuratezza $[0.7, 1.0]$, con degradazione modesta anche a $a = 0.5$ e nessun raggiungimento del punto di *break-even* rispetto a FEDAVG entro l'intervallo esplorato (RQ2).
- Sensitivity contenuta agli iperparametri di salvaguardia *cap* e *time_decay*: sulla griglia 3×3 esplorata nessuna configurazione risulta dominante entro la varianza sui tre seed, e il *default* scelto a priori cade sulla media dei punti esplorati (RQ3).

Realizzativo. La pipeline NLP a quattro stadi (NER geografico, mapping località→client, sentiment/severity hybrid, aggregazione per client) è stata implementata come sistema modulare e valutata *end-to-end* in una *run* FL con `NLPUrgencyProvider` sul dataset di lamentele realistico (sezione 6.4). Il valore di critical_ap50 ottenuto (0.104) è coerente con la caratterizzazione sistematica dello sweep, confermando che il ponte tra il modulo NLP e il modulo FL funziona nella sua totalità. L'intera pipeline è resa disponibile come codice aperto per la riproducibilità dei risultati.[3]

I limiti riconosciuti (sezione 7.3) e le direzioni di estensione (sezione 8) tracciano un percorso chiaro per il consolidamento della validità esterna dei risultati. In particolare, l'integrazione con baseline Non-IID-aware (FedProx [4], SCAFFOLD [5]) e la sostituzione del dataset sintetico

[3] Repository: https://github.com/nf0101/DLNLP.

di lamentele con dati di *civic reporting* reali rappresentano le priorità più alte per un lavoro successivo.

Il valore del contributo va infine inquadrato nell'ottica del caso d'uso. Un sistema di monitoraggio urbano automatico non ha senso solo sulla base dei suoi risultati aggregati: deve rispondere agli interessi concreti dei cittadini che lo abitano. CLARA è un primo esperimento nel far dialogare esplicitamente il canale automatico (visione dai veicoli) con il canale umano (segnalazioni testuali), e i risultati mostrano che questo dialogo, anche se attualmente mediato da un modulo NLP semplice, produce benefici misurabili sulla capacità del sistema di concentrarsi sulle anomalie più critiche per la sicurezza. È un piccolo passo verso sistemi di *smart city* in cui l'intelligenza artificiale lavora *con* i cittadini, non solo *per* loro.

Riferimenti bibliografici

[1] Deeksha Arya, Hiroya Maeda, Sanjay Kumar Ghosh, Durga Toshniwal, and Yoshihide Sekimoto. Rdd2022: A multi-national image dataset for automatic road damage detection. https://doi.org/10.6084/m9.figshare.21431547, 2022.

[2] Deeksha Arya, Hiroya Maeda, Sanjay Kumar Ghosh, Durga Toshniwal, and Yoshihide Sekimoto. Crowdsensing-based road damage detection challenge (crddc'2022). In *2022 IEEE International Conference on Big Data (Big Data)*, pages 6378–6386. IEEE, 2022.

[3] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.

[4] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine learning and systems*, volume 2, pages 429–450, 2020.

[5] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning*, pages 5132–5143. PMLR, 2020.

[6] Vito Walter Anelli, Yashar Deldjoo, Tommaso Di Noia, and Antonio Ferrara. Prioritized multi-criteria federated learning. *Intelligenza Artificiale*, 2021. arXiv:2007.08893.

[7] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, et al. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.

[8] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023.

[9] Poonam Kumari Saha, Deeksha Arya, and Yoshihide Sekimoto. Federated learning–based global road damage detection. *Computer-Aided Civil and Infrastructure Engineering*, 39(14):2223–2238, 2024.

[10] Constantine E. Kontokosta, Boyeong Hong, and Kristi Korsberg. Equity in 311 reporting: Understanding socio-spatial differentials in the propensity to complain. *arXiv preprint arXiv:1710.02452*, 2017.

[11] Lingjing Wang, Cheng Qian, Philipp Kats, Constantine Kontokosta, and Stanislav Sobolevsky. Structure of 311 service requests as a signature of urban location. *PLOS ONE*, 12(10), 2017.

[12] Clayton Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 8, pages 216–225, 2014.

[13] Stefano Baccianella, Andrea Esuli, and Fabrizio Sebastiani. SentiWordNet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining. In *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10)*, pages 2200–2204, 2010.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.

[15] Daniel Loureiro, Francesco Barbieri, Leonardo Neves, Luis Espinosa Anke, and Jose Camacho-Collados. Timelms: Diachronic language models from twitter. *arXiv preprint arXiv:2202.03829*, 2022.

[16] Francesco Barbieri, Jose Camacho-Collados, Luis Espinosa Anke, and Leonardo Neves. Tweeteval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1644–1650, 2020.

[17] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in python. 2020.

[18] Shubham Dwivedi, Deeksha Arya, and Yoshihide Sekimoto. Enhanced road damage detection with federated learning across diverse and heterogeneous global datasets. In *Proceedings of the 32nd ACM International Conference on Advances in Geographic Information Systems*, SIGSPATIAL '24, pages 711–712. ACM, 2024.

[19] Preeti Bansal and Durga Toshniwal. Analyzing civic complaints for proactive maintenance in smart city. In *2016 IEEE/ACIS 15th International Conference on Computer and Information Science (ICIS)*, pages 1–6. IEEE, 2016.

[20] Yachao Yuan, Zhen Yu, Yali Yuan, Xingyu Chen, Yingwen Wu, and Thar Baker. Roadfed: A multimodal federated learning system for improving road safety. *Internet of Things*, 38:101990, 2026.

[21] Keisuke Maeda, Naoki Ogawa, Takahiro Ogawa, and Miki Haseyama. Damage-level classification considering both correlation between image and text data and confidence of attention map. *Computer-Aided Civil and Infrastructure Engineering*, 40(6):764–781, 2025.

A Iperparametri completi degli esperimenti

Questa appendice riporta la configurazione completa degli iperparametri utilizzati nello sweep sperimentale della sezione 4.7 e nella prova del concetto *end-to-end* della sezione 6.4. L'obiettivo è la riproducibilità puntuale dei risultati riportati nel corpo del documento.

A.1 Iperparametri comuni a tutte le *run*

I seguenti iperparametri sono stati mantenuti costanti in tutte le $51 + 1$ *run* sperimentali (sweep + prova del concetto):

Tabella 7: Iperparametri comuni a tutte le *run*. La colonna "Valore" indica il valore usato; la colonna "Note" motiva brevemente la scelta.

Parametro	**Valore**	**Note**
Modello	YOLOv8n	Variante nano (3.2M param.), scelta *edge-first*
Pesi iniziali	`yolov8n.pt` (COCO)	Pre-addestrato, testa rimappata a 4 classi
Round federati	10	Scala *presentable* (v. sezione 4.4)
Epoche locali	1	Un'epoca per round federato
Batch size	16	Vincolo di memoria GPU
`imgsz`	416	Dimensione immagini in input
Optimizer	SGD	Default di Ultralytics per YOLOv8n
Learning rate iniziale	0.01	Default di Ultralytics
Momentum	0.937	Default di Ultralytics
Weight decay	0.0005	Default di Ultralytics
Frazione client per round	1.0	Tutti i 6 client partecipano ad ogni round
Seed sperimentali	$\{42, 123, 2024\}$	3 seed per stima varianza

Si noti che gli iperparametri di training locale (*learning rate*, *momentum*, *weight decay*) sono lasciati ai valori di default di Ultralytics per YOLOv8n. Un'ottimizzazione fine di questi iperparametri non è stata effettuata: il focus del lavoro è sul meccanismo di aggregazione FL e non sull'ottimizzazione del rilevatore.

A.2 Iperparametri specifici per strategia

A.2.1 FedAvg

Nessun iperparametro specifico oltre a quelli di tabella 7. I pesi di aggregazione sono proporzionali al volume dei dati locali dei client.

A.2.2 Oracle-Weighted

Un solo iperparametro specifico:

$$\alpha = 0.5,$$

il coefficiente di miscelazione tra pesatura volume-based e pesatura per criticità nella formula $\pi_k = \alpha \cdot \frac{n_k}{\sum_j n_j} + (1-\alpha) \cdot c_k$. Il valore $\alpha = 0.5$ realizza un peso equilibrato tra i due contributi. Non è stata effettuata un'ottimizzazione di α; il valore è stato scelto a priori come default ragionevole.

A.2.3 NLP-Weighted

NLP-Weighted eredita $\alpha = 0.5$ da Oracle-Weighted e aggiunge due iperparametri di salvaguardia:

$$cap = 0.5, \qquad time_decay = 1.00.$$

Sono i valori di default usati in tutte le *run* di RQ1 e RQ2. Il valore $time_decay = 1.00$ corrisponde a decadimento disattivato: la formula del *time decay* (sezione 3.2.5) si riduce in questo caso a un'identità e il segnale di urgenza entra nell'aggregazione senza attenuazione lungo i round. L'*ablation study* (sezione 6.3) esplora la griglia 3×3 attorno a questi valori, attivando progressivamente il decadimento.

A.2.4 `NoisyOracleUrgencyProvider` (usato in RQ2)

Un solo iperparametro specifico:

$$a \in \{1.0, 0.9, 0.8, 0.7, 0.5\},$$

l'accuratezza NLP simulata. Il seed del generatore di permutazioni è sincronizzato con il seed sperimentale globale in ogni *run*.

A.3 Iperparametri della pipeline NLP

Gli iperparametri interni al modulo NLP (sezione 3.3) sono riportati nella tabella 8. Come per gli iperparametri di training, non sono stati oggetto di ottimizzazione sistematica: sono stati scelti sulla base di considerazioni concettuali e valutati sul dataset di lamentele sintetiche.

Tabella 8: Iperparametri della pipeline NLP. La colonna "Sezione" indica dove il parametro è discusso nel corpo del documento.

Stadio	Parametro	Valore	Sezione
1 (NER)	Modello spaCy	`en_core_web_lg`	sezione 3.3.1
	Etichette accettate	`{GPE, LOC, FAC}`	
	Priorità entità	GPE > LOC > FAC	
2 (Mapping)	Strategia	Ibrida (NER + *pattern-match*)	sezione 3.3.2
3 (Sentiment)	Modello Transformer	`cardiffnlp/twitter-roberta-base-sentiment-latest`	sezione 3.3.3
	Peso Transformer w_T	0.4	
	Peso keyword high	+0.30 (max)	
	Peso keyword low	−0.40 (max)	
	Peso keyword medium	0.15 (*pull-to-center*)	
	Peso caps *boost*	+0.15 (max)	
	Peso excl. *boost*	+0.10 (max)	
	Soglie categoria	low $< 0.4 \leq$ med $< 0.7 \leq$ high	
4 (Aggreg.)	$u_{\min}$	0.01	sezione 3.3.4
	Normalizzazione	sum-to-1	

I pesi *keyword high*, *keyword low*, *caps* ed *excl.* sono additivi; il *medium pull* è invece il coefficiente di una combinazione convessa verso 0.5 (sezione 3.3.3).

B Statistiche complete del dataset

Questa appendice riporta le statistiche complete dei dataset utilizzati nel lavoro, sia visuale (RDD2022) sia testuale (lamentele sintetiche), integrando le distribuzioni sintetiche già discusse in sezioni 4.1 e 4.6.

B.1 RDD2022: distribuzione per client

Tabella 9 estende la tabella 1 del corpo con la ripartizione train/val per classe. Tabella 10 riporta le percentuali della classe critica D40 per client, che determinano il vettore Oracle c_k (sezioni 3.2.4 e 5.3).

Tabella 9: Distribuzione delle annotazioni di RDD2022 per client, split train/val e classe. Le righe riportano il numero di istanze delle quattro classi standard del CRDDC nei due split.

Client	Split	D00	D10	D20	D40	Totale
`czech`	train	815	328	132	162	1 437
	val	173	71	29	35	308
`india`	train	1 283	55	1 668	2 631	5 637
	val	272	13	353	556	1 194
`japan`	train	3 343	3 285	5 116	1 851	13 595
	val	706	694	1 082	392	2 874
`norway`	train	7 073	1 428	386	381	9 268
	val	1 497	302	82	80	1 961
`united_states`	train	5 572	2 720	688	111	9 091
	val	1 178	575	146	24	1 923
`china_motorbike`	train	2 210	905	529	194	3 838
	val	468	191	112	41	812
Totale	train	20 296	8 721	8 519	5 330	42 866
	val	4 294	1 846	1 804	1 128	9 072

Tabella 10: Distribuzione della classe critica D40 per client, con la percentuale rispetto al totale sugli split di training e validazione combinati. La colonna c_k riporta il vettore Oracle usato in ORACLE-WEIGHTED e con `OracleUrgencyProvider`.

Client	Istanze D40 (train + val)	% del totale	c_k
`india`	3 187	49.3%	0.493
`japan`	2 243	34.7%	0.347
`norway`	461	7.1%	0.071
`china_motorbike`	235	3.6%	0.036
`czech`	197	3.1%	0.031
`united_states`	135	2.1%	0.021
Totale	**6 458**	**100%**	**1.000**

Come discusso in sezione 4.2, il partizionamento produce un'eterogeneità fortemente Non-IID: la quota di D40 in India (49.3%) è circa 23 volte quella degli Stati Uniti (2.1%). Questa disparità è ciò che rende il caso RDD2022 un banco di prova interessante per il meccanismo NLP-Weighted.

B.2 Dataset di lamentele: distribuzione

Il progetto include due varianti del dataset testuale sintetico (sezione 4.6). Tabella 11 riporta la distribuzione del dataset bilanciato; tabella 12 quella del dataset realistico.

Tabella 11: Distribuzione del dataset di lamentele sintetiche bilanciato (`synthetic_complaints.csv`). 10 lamentele per ciascuno dei 6 client; la distribuzione sui tre livelli di severity non è uniforme, con prevalenza della classe *high*.

Client	high	medium	low	Totale
`japan`	5	3	2	10
`india`	5	3	2	10
`czech`	4	3	3	10
`norway`	5	2	3	10
`united_states`	5	2	3	10
`china_motorbike`	5	3	2	10
Totale	**29**	**16**	**15**	**60**

Tabella 12: Distribuzione del dataset di lamentele sintetiche realistico (`synthetic_complaints_realistic.csv`). Numero variabile di lamentele per client, distribuito in modo correlato (non identico) alla distribuzione di D40 in RDD2022 (tabella 10).

Client	high	medium	low	Totale	% totale
`india`	11	4	2	17	28.3%
`japan`	7	4	2	13	21.7%
`norway`	1	4	4	9	15.0%
`china_motorbike`	0	4	4	8	13.3%
`united_states`	0	3	4	7	11.7%
`czech`	0	3	3	6	10.0%
Totale	**19**	**22**	**19**	**60**	**100%**

Come discusso in sezione 4.6, la distribuzione del dataset realistico è *correlata* ma non *identica* a quella di D40 in RDD2022. La correlazione mimetizza il fatto che in un deployment reale le lamentele cittadine sono un segnale imperfetto della distribuzione oggettiva delle anomalie stradali: fattori culturali, demografici e di accessibilità dei canali di segnalazione introducono uno scostamento sistematico tra "anomalie effettivamente presenti" e "anomalie segnalate".

C Tabelle complete dello sweep sperimentale

Questa appendice riporta i risultati numerici completi delle 51 *run* sperimentali per-configurazione e per-seed. Le tabelle sono organizzate per fase dello sweep (RQ1, RQ2, Ablation) e riportano sia i valori individuali che le statistiche aggregate. Include inoltre la matrice di confusione della valutazione del modulo Sentiment, richiamata in sezione 3.3.3.

C.1 RQ1: valori individuali per seed

Tabella 13: Risultati completi di RQ1 al round 10 per ogni combinazione strategia × seed. La riga in grassetto è la media ± deviazione standard sui tre seed.

Strategia	**Seed**	critical_ap50	mAP@50	mAP@50-95
FedAvg	42	0.0935	0.1557	0.0622
	123	0.0895	0.1456	0.0580
	2024	0.0902	0.1350	0.0546
	media	**0.0911 ± 0.0021**	**0.1454 ± 0.0103**	**0.0583 ± 0.0038**
Oracle-Weighted	42	0.0852	0.1380	0.0551
	123	0.0993	0.1444	0.0572
	2024	0.1054	0.1470	0.0564
	media	**0.0966 ± 0.0104**	**0.1432 ± 0.0047**	**0.0562 ± 0.0011**
NLP-Weighted (Oracle)	42	0.1108	0.1493	0.0585
	123	0.1004	0.1394	0.0539
	2024	0.1036	0.1475	0.0584
	media	**0.1050 ± 0.0053**	**0.1454 ± 0.0053**	**0.0569 ± 0.0026**

C.2 RQ2: valori individuali per seed e livello di rumore

Tabella 14: Risultati completi di RQ2 (critical_ap50 al round 10) per ogni combinazione accuratezza a × seed. La colonna "media" riporta media ± deviazione standard sui tre seed.

Accuratezza a	**Seed 42**	**Seed 123**	**Seed 2024**	**media ± std**
1.0	0.1068	0.1036	0.0931	0.1012 ± 0.0071
0.9	0.0897	0.1005	0.1121	0.1008 ± 0.0112
0.8	0.1142	0.0974	0.0921	0.1013 ± 0.0115
0.7	0.0963	0.1048	0.1054	0.1022 ± 0.0051
0.5	0.0985	0.0767	0.1122	0.0958 ± 0.0179

C.3 Ablation: valori per configurazione

Tabella 15: Risultati completi dell'ablation: critical_ap50 al round 10 per ogni combinazione (*cap*, *time_decay*) mediata sui tre seed. La configurazione di default è in grassetto.

cap	time_decay	media ± std	# seed
0.3	0.85	0.0899 ± 0.0102	3
0.3	0.95	0.1001 ± 0.0024	3
0.3	1.00	0.0968 ± 0.0123	3
0.5	0.85	0.0938 ± 0.0086	3
0.5	0.95	0.1030 ± 0.0013	3
0.5	**1.00**	$\mathbf{0.0967 \pm 0.0077}$	3
0.7	0.85	0.0967 ± 0.0040	3
0.7	0.95	0.0989 ± 0.0031	3
0.7	1.00	0.0936 ± 0.0036	3

C.4 Prova del concetto: dettaglio della *run*

Tabella 16: Metriche per-round della *run end-to-end* di sezione 6.4: NLP-Weighted con `NLPUrgencyProvider` su `synthetic_complaints_realistic.csv`, seed=42.

Round	critical_ap50	mAP@50	mAP@50-95
1	0.000	0.026	0.010
2	0.033	0.083	0.032
3	0.052	0.099	0.039
4	0.066	0.120	0.049
5	0.073	0.130	0.052
6	0.094	0.142	0.058
7	0.089	0.154	0.061
8	0.096	0.151	0.063
9	0.100	0.154	0.061
10	0.104	0.152	0.060

C.5 Valutazione del modulo Sentiment/Severity

Tabella 17 riporta la matrice di confusione ottenuta valutando il modulo di sentiment/severity classification (sezione 3.3.3) sul dataset di lamentele sintetiche bilanciato (sezione 4.6). La configurazione finale (peso Transformer $w_T = 0.4$, peso keyword low a -0.40) ottiene un'accuratezza dell'86.7% (52 su 60 lamentele classificate correttamente).

Tabella 17: Matrice di confusione del modulo Sentiment/Severity. Le righe sono le classi vere (`expected_severity`), le colonne sono quelle predette.

Vera \ Predetta	low	medium	high	supporto
low	8	7	0	15
medium	0	16	0	16
high	0	1	28	29
previsioni	8	24	28	60

Tabella 18: Metriche per-classe del modulo Sentiment/Severity.

Classe	Precision	Recall	F1	Supporto
low	1.000	0.533	0.696	15
medium	0.667	1.000	0.800	16
high	1.000	0.966	0.982	29
macro avg	0.889	0.833	0.826	60
weighted avg	0.911	0.867	0.862	60

Sette degli otto errori sono lamentele *low* classificate come *medium*; il rimanente è una lamentela *high* classificata come *medium*. Nessun errore grave (nessuna lamentela *high* classificata come *low* o viceversa) è stato riscontrato: tutti gli errori sono di una sola categoria e tutti nella direzione del centro della scala, coerentemente con il meccanismo di *pull-to-center* descritto in sezione 3.3.3. La precision è perfetta su *low* e *high*, con debolezza localizzata sul recall di *low* (0.533).

C.6 Note sulla varianza stocastica

Come discusso in sezione 4.5 e sezione 6.1, Ultralytics introduce una varianza stocastica intrinseca nel training che, anche con seed fissato, non può essere completamente eliminata. Ci sono almeno tre sorgenti di rumore residuo:

- Operazioni CUDA non-deterministiche (in particolare nella *backward* di alcuni layer di convoluzione, non garantite bit-identiche neanche con `torch.use_deterministic_algorithms(True)` per YOLOv8n).
- Ordine di aggregazione dei gradienti quando si esegue in modalità Ray simulation (sezione 4.3), che dipende dall'ordine di completamento degli attori.
- Componenti stocastiche nella pipeline di data augmentation di Ultralytics (*mosaic*, *mixup*, *HSV* jitter), non completamente controllate dal seed globale.

La deviazione standard tipica osservata nelle *run* ripetute (dell'ordine di $\sigma \sim 0.005$–0.010 per critical_ap50) è compatibile con queste sorgenti. Le tre repliche con seed diversi $\{42, 123, 2024\}$ producono così sia varianza attribuibile all'inizializzazione dei pesi che varianza da rumore residuo del training.